OPENTYREF1 – A DATA-DRIVEN MOBILE PLATFORM FOR FORMULA 1 TELEMETRY ANALYSIS AND RACE-STRATEGY SIMULATION

A THESIS

Submitted by

Ankit Sawalakhe

for the award of the degree

of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING

Under the guidance of

Mrs. Prachi Jain



COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF ENGINEERING AND TECHNOLOGY

G H Raisoni University, Amravati

2026-27

DECLARATION

We declare that,

a. The work contained in the Report is original and has been done by ourselves under the general supervision of our supervisor.
b. We have followed the guidelines provided by the University in writing the Report.
c. We have conformed to the norms and guidelines given in the Ethical Code of Conduct of the University.
d. Whenever we have used materials (data, theoretical analysis, and text) from other sources, we have given due credit to them by citing them in the text of the Report and giving their details in the references.
e. The Report has been subjected to plagiarism check using professional software and found to be within the limits specified by the University.
f. The work has not been submitted to any other Institute for any degree or diploma.

Name of Students	**Enrollment No.**
Ankit Sawalakhe	GHRUA23011450002

CERTIFICATE

This is to certify that the report entitled

OpenTyreF1 – A Data-Driven Mobile Platform for Formula 1 Telemetry Analysis and Race-Strategy Simulation

is a bonafide work and it is submitted to

G H Raisoni University, Amravati

by,

Ankit Sawalakhe

For the partial fulfilment of the requirement in the degree of Bachelor of Technology in Computer Science & Engineering, during the academic year 2026-27.

Mrs. Prachi Jain	**Dr. Suresh Salankar**
Project Guide	**Dean, Department of CSE**

PROCEEDINGS OF EXTERNAL VIVA-VOCE EXAMINATION

The viva-voce examination of Ankit Sawalakhe on this B.Tech dissertation titled **OpenTyreF1 – A Data-Driven Mobile Platform for Formula 1 Telemetry Analysis and Race-Strategy Simulation** was conducted in the School of Engineering and Technology, in the Department of Computer Science & Engineering at G H Raisoni University, Amravati on _____.

During the viva-voce examination following examiners were present.

1. **Guide: Mrs. Prachi Jain**
2. **Ext. Examiner: _____**

Based on the presentation given by Ankit Sawalakhe both the examiners approve the dissertation for the partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science & Engineering.

Signature:

Project Guide: _____

External Examiner: _____

ACKNOWLEDGEMENT

I would like to acknowledge our deep sense of gratitude to our project guide Mrs. Prachi Jain, Department of Computer Science & Engineering, G H Raisoni University, Amravati, for her constant valuable guidance and encouragement. She gladly accepted all the pains in going through our work again and again and gave me the opportunity to learn essential research skills. This Report would not have been possible without her insightful and critical suggestions, her active participation in constructing the right models and her very supportive attitude. I will always remain grateful to her for giving direction to my life.

I express my sincere thanks to the Honourable Vice-Chancellor, the Dean-Department of CSE, Project In-charge, and my Guide for providing the necessary facilities for carrying out the research work.

I would like to acknowledge the support of my parents and siblings for their continuing support and encouragement.

Name of Students

Ankit Sawalakhe

CONTENTS

Chapter	Illustration	Page
	List of Tables	**i**
	List of Figures	**ii**
	List of Photographs (Screenshots)	**iii**
	List of Symbols and Abbreviations	**iv**
	Abstract	**v**
1.	**INTRODUCTION**	**1**
	1.1 Introduction to Formula 1 Telemetry and Race Strategy	1
	1.2 Motivation for Data-Driven Strategy Analysis	3
	1.3 Objectives of OpenTyreF1	3
	1.4 Scope of the Project	4
	1.5 Key Contributions	4
	1.6 Organisation of the Report	5
2.	**LITERATURE SURVEY**	**6**
	2.1 Overview of Existing Formula 1 Analytics Systems	6
	2.1.1 Public tools mostly describe the past	6
	2.1.2 Team software is powerful but closed	6
	2.1.3 The gap that still exists	6
	2.2 Race-Strategy Simulation Research	6
	2.3 Machine Learning for Lap-Time Prediction	7
	2.4 Open Data Sources for Formula 1	7
	2.5 Limitations of Existing Systems	7
	2.6 Comparison of Existing Approaches with OpenTyreF1	9
	2.7 Research Gap and Problem Statement	9
3.	**METHODOLOGY**	**11**
	3.1 Introduction	11
	3.2 Overview of Methodology	12
	3.2.1 Phase 1: Requirement analysis	13
	3.2.2 Phase 2: System design.	13
	3.2.3 Phase 3: Data engineering	15
	3.2.4 Phase 4: Model development	18
	3.2.5 Phase 5: Implementation	18
	3.2.6 Phase 6: Testing and deployment	18
	3.3 Advantages of the Methodology	19

3.4 System Workflow 20
3.5 Tools and Technologies Used 20
3.6 System Architecture 20
3.6.1 Presentation layer (mobile app) 23
3.6.2 Application layer (backend) 23
3.6.3 Machine learning and simulation layer 24
3.6.4 Data layer (database) 24
3.6.5 Integration layer (external services) 24
3.7 Core Algorithms and Modules 24
3.7.1 Clean-lap filter 24
3.7.2 Lap-time prediction model 26
3.7.3 Strategy simulation module 27
3.7.4 Monte Carlo outcome module 28
3.7.5 Pit-wall decision modules 29
3.7.6 Grounded assistant module 30
3.8 Error Handling and Data Integrity 30

4. STRENGTH & LIMITATION 32
4.1 Strengths 32
4.2 Limitations 33
4.3 Use Cases for Software 33

5. RESULT & DISCUSSION 35
5.1 System Performance 35
5.2 Functional Testing 35
5.3 Prediction Accuracy 35
5.4 Target Design versus Feature Richness 36
5.5 Fuel-Corrected Tyre Degradation 38
5.6 User Experience 39
5.7 Discussion 39
5.8 Results 40
5.9 Screenshots 40

6. CONCLUSIONS AND SCOPE FOR FUTURE WORK 43
6.1 Conclusion 43
6.2 Future Enhancements 44
6.3 Final Remarks 45

REFERENCES 46

RESEARCH PAPER PUBLICATION 47

CERTIFICATES OF RESEARCH PUBLICATION 48

LIST OF TABLES

Table	Title	Page No
1.1	Key Formula 1 Strategy Terms	2
2.1	Literature Review	8
2.2	Comparison of Existing Approaches	9
3.1	Overview of Methodology	12
3.2	Hardware and Software Requirements	13
3.3	Database Tables	17
3.4	Data Sources and Coverage	18
3.5	Types of Testing	19
3.6	Tools and Technologies	22
3.7	Main REST API Endpoints	23
3.8	XGBoost Configuration	26
3.9	Pit-Wall Strategy Modules	29
5.1	Representative Test Cases	36
5.2	Holdout Performance (12 Unseen Races)	36
5.3	Objectives and Achieved Outcomes	40
6.1	Future Enhancement Roadmap	44

LIST OF FIGURES

Figure	Title	Page No
3.1	Hybrid SDLC Model Followed in the Project	11
3.2	Project Timeline	12
3.3	Use Case Diagram	14
3.4	Data Flow Diagram - Level 0	14
3.5	Data Flow Diagram - Level 1	15
3.6	Lap Records by Season and Source	16
3.7	Entity-Relationship Diagram of the Database	16
3.8	Background Data-Sync Flowchart	17
3.9	Deployment Diagram	19
3.10	System Workflow Diagram	21
3.11	Application Navigation Map	21
3.12	System Architecture Diagram	22
3.13	Track-Status State Machine	25
3.14	Clean-Lap Admission Cascade	25
3.15	Lap-Time Distribution Before and After Filtering	26
3.16	Data and Model-Training Pipeline	27
3.17	Sequence Diagram of a Strategy Simulation	28
3.18	Monte Carlo Outcome Procedure	29
3.19	Grounded Assistant Pipeline	30
5.1	Holdout Error of the Model against Both Baselines	37
5.2	Ablation - Target Parameterisation vs. Feature Set	37
5.3	XGBoost Gain Importance of the Six Features	38
5.4	Lap Records by Tyre Compound	38
5.5	Median Fuel-Corrected Degradation by Compound	39

LIST OF PHOTOGRAPHS (SCREENSHOTS)

Screenshot	Title	Page No
5.1	Live Tab - Next Race Countdown	41
5.2	Latest Result and Timing Tower	41
5.3	Track Map and Conditions	41
5.4	Drivers List	41
5.5	Races List	41
5.6	Race Pace and Positions	41
5.7	Telemetry Comparison	42
5.8	Tyre Degradation Analysis	42
5.9	Strategy Tab - Module Picker	42
5.10	Stint Planner	42
5.11	Simulation Result	42
5.12	Grounded Assistant	42

LIST OF SYMBOLS AND ABBREVIATIONS

Symbol / Abbreviation	Description
y	Actual lap time, s
$\hat{y}$	Predicted lap time, s
m	Median clean lap time of a race, s
$f(\mathbf{x})$	Learned lap-time offset from the race median
n	Number of laps evaluated
R^2	Coefficient of determination
s	Second
API	Application Programming Interface
DFD	Data Flow Diagram
DNF / DNS / DSQ	Did Not Finish / Did Not Start / Disqualified
DRS	Drag Reduction System
ERS	Energy Recovery System
F1	Formula 1
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
MAE	Mean Absolute Error
REST	Representational State Transfer
RMSE	Root Mean Square Error
RSS	Resident Set Size (memory)
SC	Safety Car
SDLC	Software Development Life Cycle
SHAP	Shapley Additive Explanations
SQL	Structured Query Language
TTL	Time To Live (cache)
UI	User Interface
VSC	Virtual Safety Car
XGBoost	Extreme Gradient Boosting

ABSTRACT

Formula 1 publishes one of the richest open telemetry streams in professional sport, yet most public tools built on it are descriptive: they show what happened and stop there. This project presents OpenTyreF1, a full-stack system that turns that data into decision support on a mobile phone. The system integrates three public sources (Jolpica-F1/Ergast, OpenF1 and FastF1) into one normalised 15-table relational corpus of 237,308 records covering 492 races from 2002 to 2026. On top of this corpus it builds a shared clean-lap filter, a gradient-boosted (XGBoost) lap-time model, a counterfactual stint-strategy simulator, a 1,500-run Monte Carlo outcome estimator, seven pit-wall decision modules and a grounded question-answering assistant, all served by a FastAPI backend to an Expo / React Native app.

Evaluated on 12 completely unseen races, the model achieves a mean absolute error of 0.893 s, RMSE of 1.205 s and R^2 of 0.978, beating the strongest naive baseline by 0.270 s. An ablation shows that predicting a lap's offset from its race median, rather than the absolute lap time, cuts error four-fold, a far larger effect than adding features. The results show that, for motorsport data of this size, careful lap admission, target design and leakage-free evaluation matter more than the choice of learner. The deployed backend runs in about 204 MB of memory on free-tier hosting, making the system practical, reproducible and inexpensive to operate.

Keywords: Formula 1, Telemetry Analytics, Tyre Degradation, XGBoost, Race-Strategy Simulation, Monte Carlo Methods, Data Leakage, Mobile Decision Support, FastAPI, React Native

INTRODUCTION

Formula 1 is one of the most data-intensive sports in the world. A modern car carries around three hundred sensors, and a meaningful part of that stream (lap times, tyre compounds and ages, pit-stop durations, race-control messages and trackside weather) is now published openly through community APIs. OpenTyreF1 is a full-stack system that turns this public data into decision support on a mobile phone. It lets a user open any replayable race, build a tyre strategy, run it against what really happened and receive an answer grounded in real lap data instead of generated text.

1.1 Introduction to Formula 1 Telemetry and Race Strategy

In Formula 1, the difference between winning and finishing outside the points is often decided not on the track but on the pit wall. Teams employ dedicated strategy engineers who decide when a car should stop for new tyres, which compound it should fit and how to react when a safety car is deployed. These decisions are driven by data: how quickly each tyre compound loses grip, how much time a pit stop costs at a particular circuit, how fast the car becomes as fuel burns off and how likely an overtake is at a given gap.

For a long time this data was locked inside the teams. That has changed. Projects such as the Ergast API and its successor Jolpica-F1 publish historical results going back decades, OpenF1 exposes live timing, stints, pit stops and weather from 2023 onwards, and the FastF1 Python library reconstructs lap data from the official timing service back to 2018. The raw material for quantitative race analysis is therefore no longer scarce. What remains scarce is the analytical layer that turns it into decisions.

Key challenges in working with public Formula 1 data include:

- **Fragmented sources:** The three main public sources have complementary coverage but no shared identifier scheme, so the same race, driver or session is named differently in each one.
- **Noisy lap data:** Raw lap times include standing starts, in-laps and out-laps, safety-car periods and outliers. Training a model on raw laps teaches it pit stops and safety cars rather than tyre behaviour.
- **Rate limits:** Free public APIs such as OpenF1 allow only about three requests per second, so a mobile app cannot query them directly for every user.
- **Descriptive-only tools:** Most fan tools plot what happened (a position chart, a stint diagram, a table of pit stops) and stop there.

- **Evaluation pitfalls:** Laps from the same race share a car, a fuel load and a weather trace, so naive random splits leak information and overstate model accuracy.

These issues together mean that, although the data is open, very few people outside the teams can use it to answer the question a strategist actually asks.

Several terms from race strategy are used throughout this report. Table 1.1 defines them briefly so that the later chapters can be read without a motorsport background.

Table 1.1: Key Formula 1 Strategy Terms

Term	Meaning
Stint	The consecutive laps a driver completes on one set of tyres, between the start, pit stops and the finish
Tyre compound	The rubber specification of a tyre: soft, medium and hard for dry running, intermediate and wet for rain
Tyre degradation	The loss of lap time as a tyre ages, expressed in seconds per lap
Pit loss	The total time lost by entering the pit lane, stopping and rejoining, compared with staying on track
Undercut / overcut	Gaining track position on a rival by pitting before (undercut) or after (overcut) them
Safety car (SC)	A neutralisation in which the field follows a safety car at reduced speed, making pit stops cheaper
Virtual safety car (VSC)	A neutralisation in which every driver must respect a reduced lap-time delta without a physical safety car
Clean lap	A lap that reflects true racing pace: not the opening lap, not affected by a pit stop, not neutralised and within 107% of the driver's median
Fuel effect	The lap-time gain as fuel burns off during a race, which must be separated from tyre wear
DRS	Drag Reduction System: a movable rear-wing flap that helps a following car to overtake
Counterfactual simulation	Replaying a real race with a different strategy for one driver while every rival keeps their real lap times

1.2 Motivation for Data-Driven Strategy Analysis

The question a race strategist asks is counterfactual: "If this driver had pitted three laps later on the hard tyre, where would they have finished, and how sure can we be?" Answering it needs a calibrated lap-time model, an explicit rule for which laps represent true racing pace, a simulator that compares a hypothetical run against the field's real behaviour, and an honest statement of uncertainty.

The motivation behind developing OpenTyreF1 is driven by several factors:

- **From description to decision support:** Users should be able to test an alternative strategy, not only look at the one that was used.
- **Grounded answers:** Every number shown to the user should come from the database or the simulator, never from free-form generated text that may be fluent but wrong.
- **Honest evaluation:** A lap-time model is only useful if its error is measured on races it has never seen. A leakage-free protocol was therefore a design requirement from the start.
- **Accessibility:** A mobile application puts telemetry, race replay and strategy simulation in the hands of fans, students and analysts without any installation or programming.
- **Low running cost:** The system had to run on free-tier hosting, which shapes decisions such as caching telemetry on demand instead of bulk-storing it.

Furthermore, building such a system is a demanding exercise in practical data engineering and machine learning. It brings together data ingestion, relational modelling, model training, simulation, API design and mobile development in a single coherent product.

1.3 Objectives of OpenTyreF1

The key objectives of this project are:

- To integrate heterogeneous public Formula 1 data sources into a single, reproducible relational corpus with idempotent, rate-limit-safe ingestion.
- To define one shared clean-lap filter so that model training, published analytics and the mobile app always agree on what a representative lap is.
- To train a lap-time prediction model and evaluate it with a leakage-free protocol that holds out whole races rather than random laps.
- To build a counterfactual strategy simulator that converts predicted lap times into a finishing position against the real field.
- To quantify uncertainty with Monte Carlo simulation and provide bounded pit-wall decision modules backed by historical base rates.
- To deliver all of this through a fast, cross-platform mobile application with a grounded

natural-language assistant.

1.4 Scope of the Project

OpenTyreF1 is designed not just as a viewer of Formula 1 data, but as a decision-support system for race strategy. The platform covers 492 races from 2002 to 2026 stored in a normalised 15-table relational database of 237,308 records. On top of this corpus it provides a live feed of the latest session, a circuit dossier traced from real telemetry coordinates, driver career statistics, race replay, pace charts, fuel-corrected tyre degradation, two-driver telemetry comparison, a stint-strategy simulator with seven pit-wall modules, and a grounded question-answering assistant.

From a system architecture perspective, OpenTyreF1 operates through a three-layer design: an Expo / React Native mobile client, a FastAPI backend that hosts the analytics and machine-learning services, and a data layer that combines the relational store with external APIs reached through a cache. Car telemetry and track coordinates are fetched on demand rather than bulk-stored, because a single race alone is millions of rows.

Equally important is what OpenTyreF1 intentionally avoids. It does not attempt to replace proprietary team software or simulate every car in the field. The simulator is deliberately single-agent: it predicts only the subject driver's laps and holds rivals at their observed lap times, anchoring every counterfactual to a race that really happened. The model is trained on 2023 to 2026 race laps, and the assistant only answers questions that its data can support.

The project is also scoped as an engineering exercise in reproducibility. The database is committed with the code, every ingestion step can be re-run safely, the model artifact is stored together with its measured metrics, and the evaluation protocol is fixed in code. Anyone who clones the repository can therefore start the backend, open the app and obtain the same numbers that are reported in this thesis.

1.5 Key Contributions

The main contributions of this project are:

- **A unified open corpus:** 237,308 records from Jolpica-F1, OpenF1 and FastF1 reconciled into one 15-table schema with a documented alias map and logged skips instead of silent guesses.
- **A single clean-lap definition:** one filter shared by training, analytics and the API, with neutralised periods reconstructed from race-control messages.
- **An honestly evaluated lap-time model:** XGBoost on a race-relative target, scored once on 12 unseen races, reaching 0.893 s MAE and beating the matched baseline by

0.270 s.

- **A counterfactual simulator with uncertainty:** a five-step strategy simulator, a 1,500-run Monte Carlo estimator and seven pit-wall modules that always report their sample size.
- **A grounded mobile experience:** a cross-platform app and an assistant whose every answer cites the data it was built from.

Together these contributions show that serious strategy analysis does not require private team data or expensive infrastructure, provided the data is cleaned carefully and the evaluation is honest.

1.6 Organisation of the Report

The rest of this report is organised as follows:

- **Chapter 2 – Literature Survey** surveys existing Formula 1 analytics tools, race-strategy simulation research, machine learning for lap-time prediction and the open data sources used, and identifies the gap that OpenTyreF1 addresses.
- **Chapter 3 – Methodology** describes the development phases, requirements, UML and data-flow diagrams, the database design, the system architecture and the core algorithms: the clean-lap filter, the lap-time model, the simulator, the Monte Carlo module, the pit-wall modules and the assistant.
- **Chapter 4 – Strength & Limitation** discusses where the system is strong, where it is limited and who can use it.
- **Chapter 5 – Result & Discussion** reports system performance, functional testing, prediction accuracy, the ablation study, tyre-degradation findings and screenshots of the application.
- **Chapter 6 – Conclusions and Scope for Future Work** summarises the work and outlines the next steps.

LITERATURE SURVEY

2.1 Overview of Existing Formula 1 Analytics Systems

2.1.1 Public tools mostly describe the past

A growing number of websites, notebooks and apps visualise Formula 1 data. Typical outputs are position charts, lap-time scatter plots, stint diagrams and tables of pit stops. These tools are useful for understanding what happened in a race, and libraries such as FastF1 have made such plots easy to produce. However, they are descriptive: they show the strategy that was used, but they cannot tell the user what would have happened under a different one.

2.1.2 Team software is powerful but closed

Formula 1 teams run sophisticated strategy simulators in their factories and on the pit wall. These systems model tyre wear, fuel, traffic and safety-car probabilities and are run thousands of times before and during each race. They are, however, proprietary, tuned to private telemetry and unavailable to students, researchers or fans.

2.1.3 The gap that still exists

Between these two extremes there is a clear gap. There is open data rich enough to support counterfactual strategy analysis, and there are well-known statistical and machine-learning methods to do it, but there is no open, end-to-end system that reconciles the public sources, measures its own accuracy honestly and puts the result in the hands of a user on a phone. OpenTyreF1 is built to fill that gap.

2.2 Race-Strategy Simulation Research

Race-strategy simulation for Formula 1 was formalised by Bekker and Lotz [1], who modelled a race as a sequence of stochastic events and compared pit strategies through repeated discrete-event simulation. Heilmeier et al. [2] developed a lap-wise race simulation that treats fuel mass, tyre degradation and pit-lane loss as separable additive terms, and later replaced hand-tuned degradation curves with a neural “virtual strategy engineer” trained on historical laps [3]. Liu and Fotouhi [4] applied Monte Carlo tree search to Formula E strategy.

These simulators are generative: they model every car in the field, which allows multi-agent what-if analysis but makes the output sensitive to every rival’s parameters. Open-

TyreF1 is deliberately semi-parametric. It predicts only the subject driver's laps and holds rivals at their observed lap times, so every counterfactual stays anchored to a real race.

2.3 Machine Learning for Lap-Time Prediction

For the learner, gradient-boosted decision trees [5] and their XGBoost implementation [6] remain the standard for heterogeneous tabular regression. Systematic comparisons continue to show tree ensembles matching or beating deep networks on tabular data of this scale [7]. Lap-time data is exactly this kind of problem: a mix of categorical features (compound, circuit, driver) and numeric features (tyre age, lap number, track temperature) with a few tens of thousands of rows.

The hazard of evaluating dependent samples with independent-sample methods is well documented. Kaufman et al. [8] catalogue leakage in data mining, and Roberts et al. [9] formalise blocked cross-validation for structured data. In motorsport, laps from the same race share a car, a fuel load, a track surface and a weather trace. A whole-race split, in which the race is the block, is therefore the motorsport analogue of blocked cross-validation and is the protocol adopted in this project.

2.4 Open Data Sources for Formula 1

On the data side, the Ergast API [10] and its successor Jolpica-F1 [11] supply historical results, grids and standings. OpenF1 [12] exposes live and historical timing from 2023, including laps, stints, pit stops, race-control messages, weather and car telemetry. FastF1 [13] reconstructs laps from the official timing service back to 2018. To our knowledge, no published system reconciles all three into one normalised corpus and serves counterfactual strategy analysis on a phone.

2.5 Limitations of Existing Systems

Existing public tools for Formula 1 analysis share several limitations. They are mostly descriptive and cannot evaluate an alternative strategy. Many of them use a single data source, so they either lack historical depth or lack lap-level detail. Those that fit models rarely state how the model was evaluated, and random lap-level splits are common, which leaks race-specific structure into the test set and overstates accuracy.

Academic simulators address strategy directly but are usually built for research rather than for end users, depend on parameters that the public cannot observe and are not delivered as an application. Proprietary team tools are accurate but closed. None of these systems combines open data integration, an honestly evaluated model, a counterfactual

Table 2.1: Literature Review

Ref.	Year	Methodology	Key finding	Limitation
[1]	2009	Stochastic discrete-event race simulation	Pit strategies compared by repeated simulation	Hand-set parameters; no learned lap model
[2]	2018	Lap-wise simulation with fuel, tyre and pit terms	Separable terms give interpretable lap times	Degradation curves tuned by hand
[3]	2020	Neural networks trained on historical laps	Learns strategy decisions from data	Needs team-grade data
[4]	2020	Neural networks with Monte Carlo tree search	Search finds energy-optimal strategies	Formula-E specific; heavy compute
[5]	2001	Gradient boosting of regression trees	Stage-wise trees give a strong tabular learner	Sensitive to tuning; prone to overfit
[6]	2016	Regularised gradient-boosted trees	State-of-the-art tabular accuracy, fast training	Feature importance hard to interpret
[7]	2022	Benchmark of trees vs. deep networks	Trees win on medium-sized tabular data	Excludes time-series structure
[8]	2011	Formal analysis of data leakage	Leakage inflates reported accuracy	General; not domain specific
[9]	2017	Blocked cross-validation	Blocking gives honest error estimates	Larger, more pessimistic error bars
[10]	n.d.	Public REST API for F1 results	Historical results, grids and standings	Discontinued; no lap-level timing
[11]	n.d.	Ergast-compatible open API	Continues Ergast data and schema	Results only; some calendar gaps
[12]	n.d.	Open live and historical timing API	Laps, stints, pits, weather, race control	Covers 2023 onwards; rate-limited
[13]	n.d.	Python parser of the official timing feed	Reconstructs laps and compounds from 2018	Slow bulk loads; heavy dependencies
[14]	2017	Shapley-value feature attribution (SHAP)	Consistent, additive explanations	Costly; correlated features share credit
[15]	1993	Bootstrap resampling	Distribution-free uncertainty estimates	Assumes independent samples
[16]	2011	Machine-learning library in Python	Uniform API for models and metrics	General-purpose; not domain specific

simulator, explicit uncertainty and a mobile interface in one product. These limitations define the requirements that OpenTyreF1 was built to meet.

2.6 Comparison of Existing Approaches with OpenTyreF1

Table 2.2 compares the main categories of existing Formula 1 analysis approaches with OpenTyreF1 across the capabilities that matter for strategy decision support. Each category is strong in some respects, but none combines all of them.

Table 2.2: Comparison of Existing Approaches

Capability	Descriptive fan tools	Academic simulators	Team software	OpenTyreF1
Open / freely available	Yes	Partly	No	Yes
Multiple data sources reconciled	Rarely	No	Yes	Yes (3 sources)
Counterfactual strategy simulation	No	Yes	Yes	Yes
Leakage-free model evaluation	N/A	Varies	Unknown	Whole-race holdout
Uncertainty estimate	No	Sometimes	Yes	Monte Carlo, 1,500 runs
Grounded natural-language answers	No	No	No	Yes
Mobile application	Some	No	Internal only	Android / iOS

2.7 Research Gap and Problem Statement

The review shows that the building blocks for open race-strategy analysis exist separately: open data sources, proven simulation approaches, strong tabular learners and well-understood evaluation methodology. What is missing is a system that joins them end to end and is honest about its own accuracy. The problem addressed by this project can therefore be stated as follows: to design and build an open, reproducible system that reconciles public Formula 1 data, learns lap times with a leakage-free evaluation, simulates alternative tyre strategies against the real field with an explicit measure of uncertainty, and delivers the results to users through a mobile application with grounded explanations.

From this problem statement, the review leads directly to five design principles that

guided every later decision in the project:

1. **Anchor to reality:** every counterfactual is built on a real race, with rivals held at their observed times, so the answer can never drift far from what actually happened.
2. **Measure before claiming:** no accuracy figure is reported unless it was measured on whole races the model has never seen, against a baseline that already knows the circuit.
3. **One definition per concept:** a clean lap, a pit loss and a neutralised lap are each defined once in code and reused everywhere.
4. **Show the evidence:** every recommendation carries its sample size and every answer its facts, so users can judge how far to trust it.
5. **Stay cheap and reproducible:** the system must run on free-tier hosting and produce the same numbers from a fresh clone.

These principles also explain what the project deliberately leaves out. It does not try to model every car, predict live races before their median is known or answer questions its data cannot support, because each of these would trade honesty for apparent capability.

To make the problem statement testable, the project was organised around four research questions:

- **RQ1:** Can three public sources with no shared identifiers be reconciled into one corpus without fabricating records?
- **RQ2:** How accurate is a lap-time model when it is evaluated on whole races it has never seen, compared with baselines that already know the circuit and the race?
- **RQ3:** Which matters more for accuracy: the parameterisation of the target, the number of features or the choice of learner?
- **RQ4:** Can counterfactual strategy results, together with their uncertainty, be delivered on a phone from free-tier hosting, with every number traceable to its data?

Chapters 3 and 5 answer these questions in turn: RQ1 through the ingestion design and the corpus statistics, RQ2 and RQ3 through the whole-race holdout evaluation and the ablation study, and RQ4 through the simulator, the Monte Carlo module, the deployment and the application screens.

METHODOLOGY

3.1 Introduction

The methodology defines the structured approach used to design, develop and deploy OpenTyreF1, a data-driven mobile platform for Formula 1 telemetry analysis and race-strategy simulation. The primary objective of this methodology is to ensure that the system is accurate, reproducible and fast enough to use on a mobile network. A hybrid Software Development Life Cycle (SDLC) approach has been adopted, combining the Waterfall model for structured planning of the data layer with the Iterative model for continuous refinement of the model, the simulator and the user interface. This allows systematic development while keeping the flexibility to improve features based on testing and measured results.

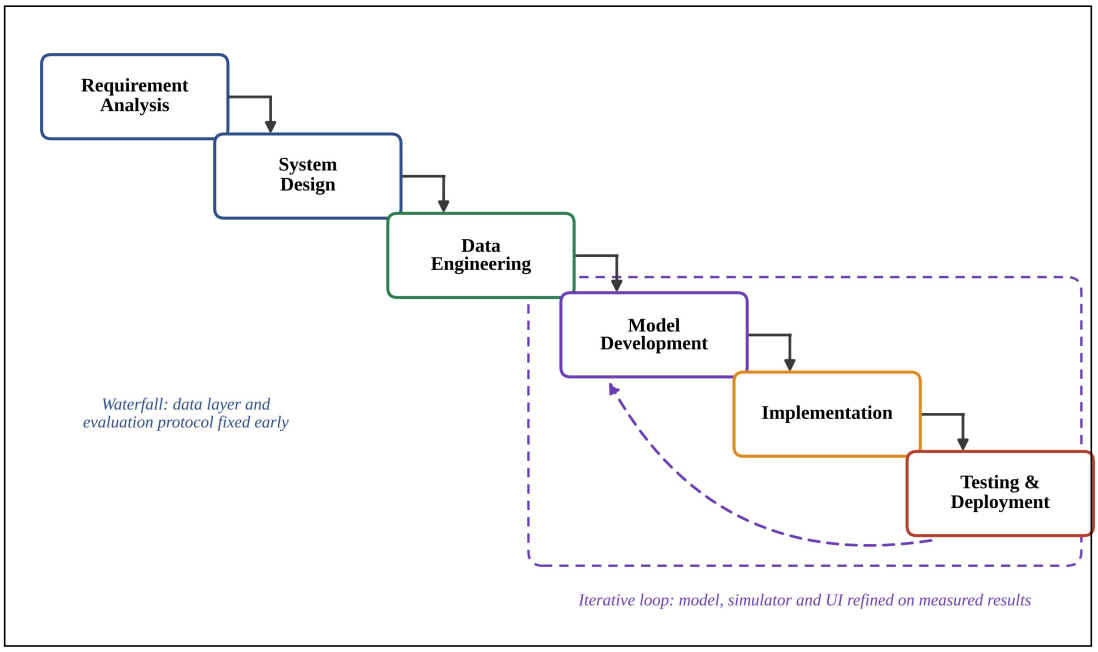


Figure 3.1: Hybrid SDLC Model Followed in the Project

As Figure 3.1 shows, the first three phases were carried out in sequence because everything later depends on them: the schema, the identifier reconciliation and the evaluation protocol had to be fixed before any model score could be trusted. The last three phases were then iterated several times, each iteration being judged against the same holdout races and the same naive baselines.

Table 3.1: Overview of Methodology

PHASE	DESCRIPTION
Phase 1: Requirement Analysis	Identifying the questions a strategist asks and the public data able to answer them.
Phase 2: System Design	Designing the three-layer architecture, the 15-table schema and the typed API contract.
Phase 3: Data Engineering	Building ingestion from Jolpica-F1, OpenF1 and FastF1 and reconciling identifiers.
Phase 4: Model Development	Defining the clean-lap filter, features, target and the leakage-free evaluation protocol.
Phase 5: Implementation	Developing the FastAPI services and the Expo / React Native client.
Phase 6: Testing and Deployment	Verifying endpoints, sync and migration; containerising the backend and building the app.

3.2 Overview of Methodology

The development of OpenTyreF1 was carried out in six major phases: requirement analysis, system design, data engineering, model development, implementation, and testing with deployment. Each phase plays a critical role in building a reliable system. The methodology follows a structured workflow in which a user request from the app is processed by backend services, combined with database records and cached external data, and returned to the user as a chart, a table or a simulation result with an explanation.

The approximate schedule of the work is shown in Figure 3.2. Several phases overlapped: backend services were developed while the model was being evaluated, and the mobile screens were built against the typed API contract as soon as each endpoint became available.

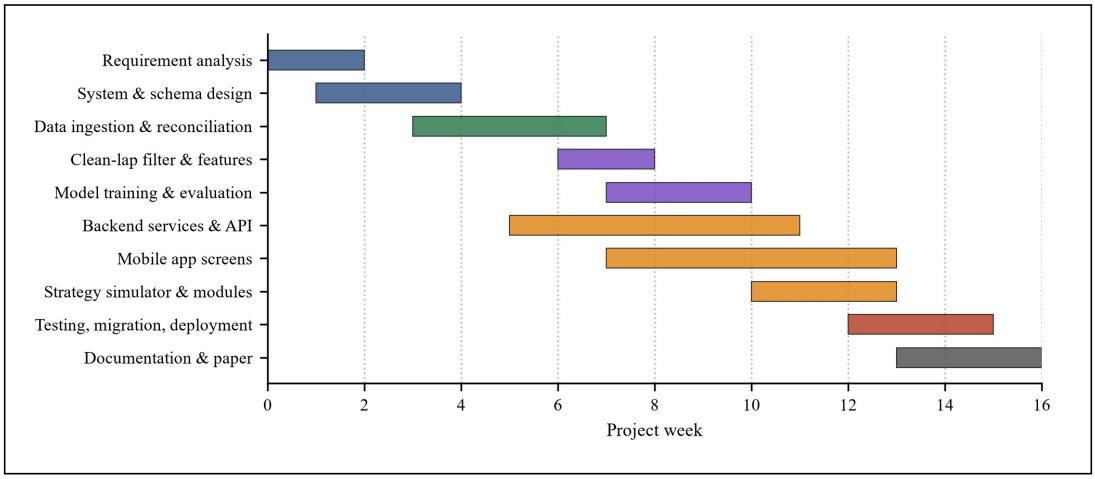


Figure 3.2: Project Timeline

3.2.1 Phase 1: Requirement analysis

The first phase focused on identifying the questions a race strategist and an informed fan actually ask: where would a driver have finished on a different strategy, how quickly does each compound degrade, when is pitting under a safety car worth it, and how likely is an overtake at a given gap. The functional requirements include browsing seasons, races and drivers, replaying a race, comparing telemetry, building a stint plan, running a simulation and asking questions in natural language. The non-functional requirements include fast response time on a mobile network, staying within upstream API rate limits, low hosting cost and reproducible results.

Table 3.2: Hardware and Software Requirements

Category	Requirement
Development machine	Windows / macOS / Linux PC, 8 GB RAM or more, 2 GB free disk space
Mobile device	Android or iOS smartphone running Expo Go or an EAS-built app
Backend runtime	Python 3.11, FastAPI, Uvicorn, SQLAlchemy, XGBoost, pandas
Mobile runtime	Node.js, Expo SDK 57, React Native 0.86, TypeScript
Database	SQLite (bundled) or PostgreSQL (Supabase)
Hosting	Docker-capable host with at least 512 MB RAM (Render free tier)
Network	Internet access for live data and on-demand telemetry

3.2.2 Phase 2: System design

In this phase the requirements were transformed into a structured design. OpenTyreF1 follows a client-server architecture. The mobile client handles interaction and rendering, and the backend hosts every computation. The phone never calls an upstream API directly; all third-party traffic is funnelled through the backend's cache. A 15-table relational schema was designed for seasons, circuits, drivers, constructors, races, sessions, results, laps, stints, pit stops, weather, race-control messages, overtakes and supporting tables, and a typed API contract was defined between the client and the server.

The functional behaviour of the system was captured with a use case diagram (Figure 3.3). The primary actor is the user of the mobile app, who can view the live feed, browse drivers and races, replay a race, compare telemetry, analyse tyre degradation, build and simulate a strategy, use the pit-wall modules and ask the assistant. Two secondary actors support the system: the external Formula 1 APIs that supply the data, and the background sync service that keeps it current.

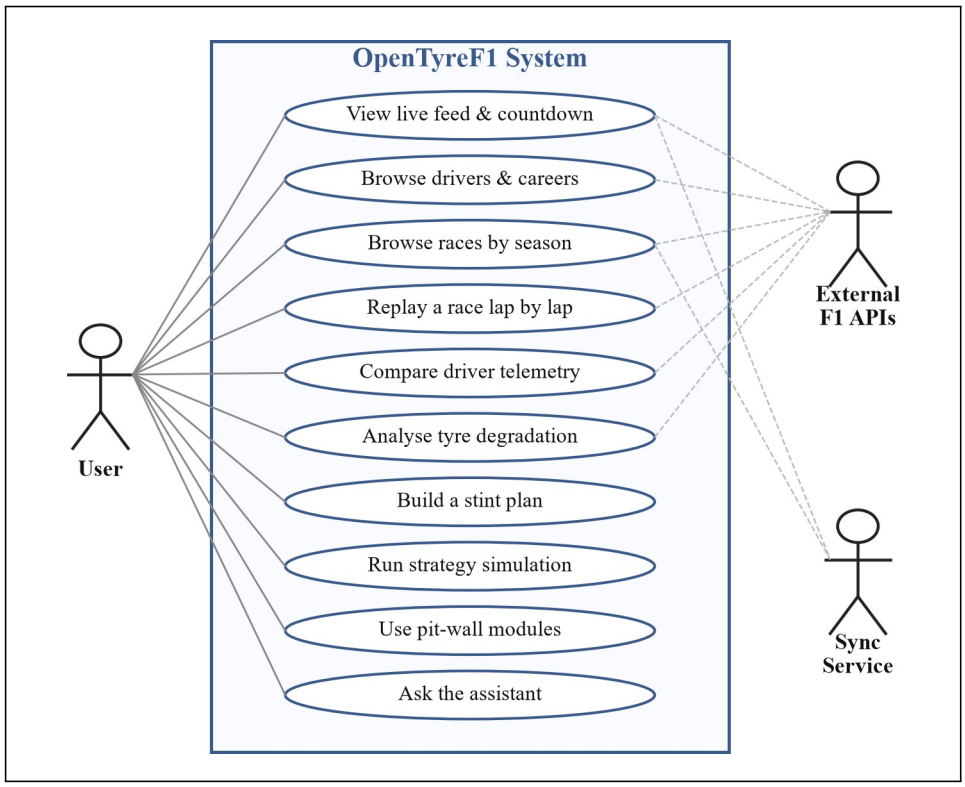


Figure 3.3: Use Case Diagram

The flow of data through the system is described with data-flow diagrams. The context-level diagram (Figure 3.4) treats OpenTyreF1 as a single process exchanging data with the user and the external sources. The level-1 diagram (Figure 3.5) decomposes it into five processes (ingestion, clean-lap filtering, model training and prediction, strategy simulation, and analytics with answers) and three data stores: the F1 database, the model artifact and the telemetry cache.

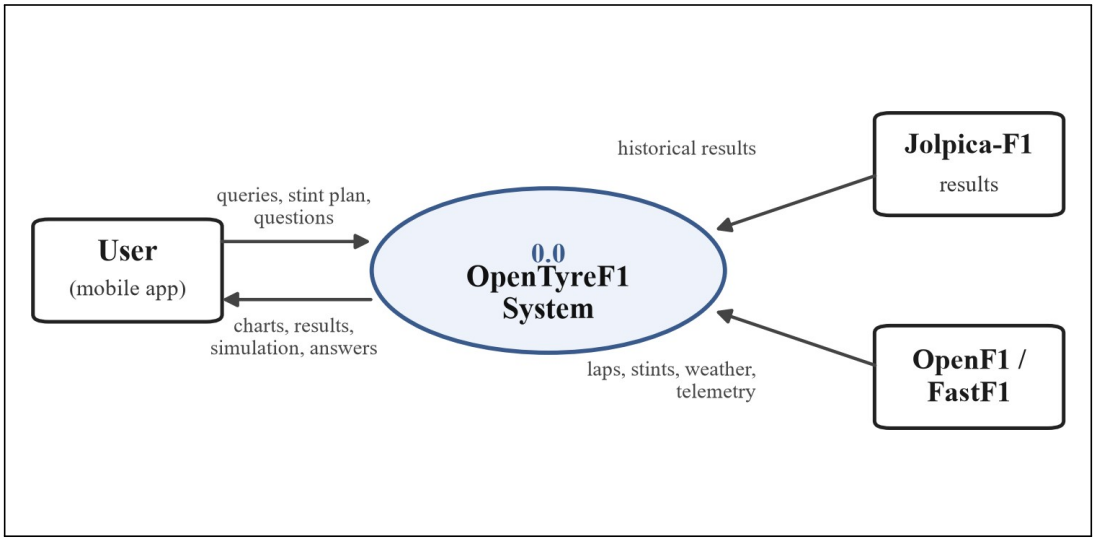


Figure 3.4: Data Flow Diagram - Level 0

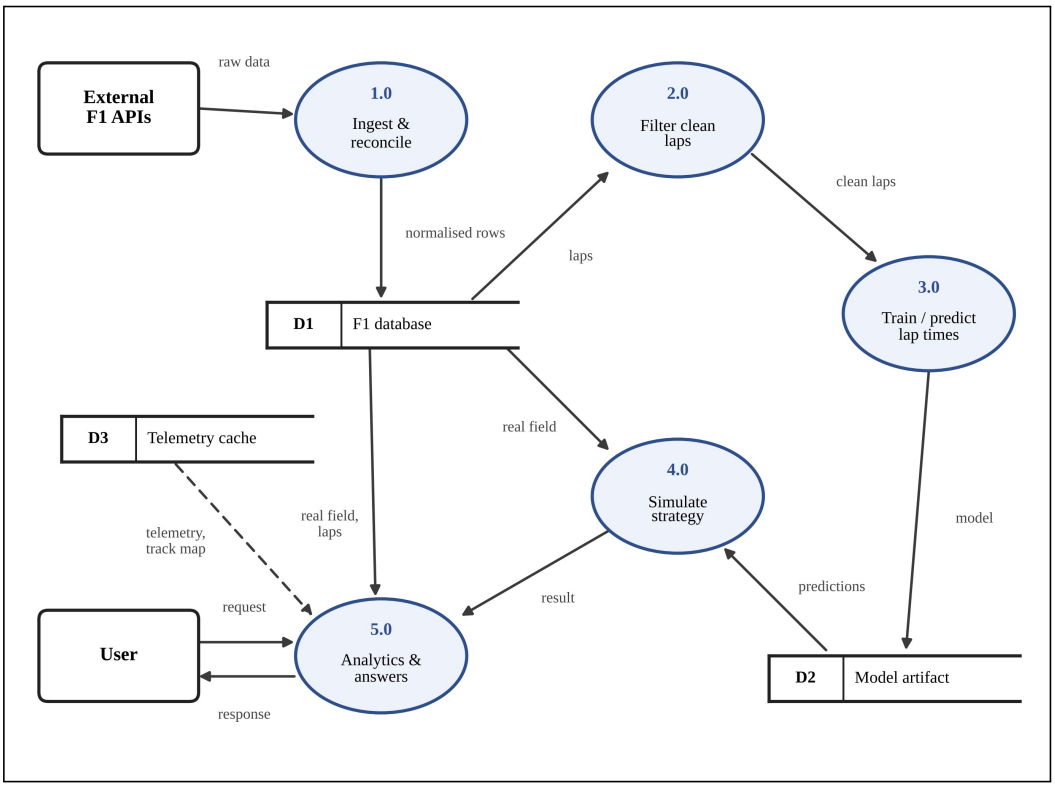


Figure 3.5: Data Flow Diagram - Level 1

3.2.3 Phase 3: Data engineering

The three sources have complementary coverage and no shared identifier scheme. Races are matched on a normalised (year, location) pair with a curated alias map. A session that matches no race is skipped with a logged reason rather than stored with a fabricated parent. This rule surfaced a real upstream gap: the Jolpica 2026 calendar omits two rounds that OpenF1 does contain. Every ingestion routine checks for existing data before making a network call, so it is idempotent and can resume safely under rate limits. The resulting corpus holds 237,308 rows, including 167,086 lap records, 22,840 reconstructed overtakes, 11,129 race-control messages and 10,068 race results.

The lap-level coverage of the corpus by season is shown in Figure 3.6. OpenF1 supplies the 2023 to 2026 seasons, while FastF1 backfills 2018 to 2022, giving between roughly 15,000 and 29,000 lap records per season.

The database design is shown in the entity-relationship diagram of Figure 3.7. The historical side of the schema is centred on races, drivers and constructors, linked through results and standings. The timing side is centred on sessions: every session belongs to a race and owns its laps, stints, pit stops, weather readings, race-control messages, results, overtakes and the list of drivers who took part. Table 3.3 summarises the purpose of each table.

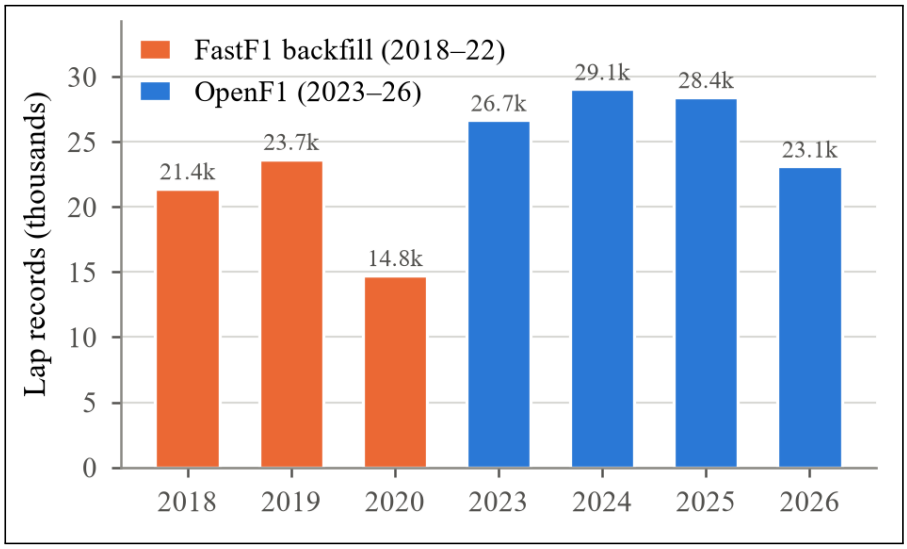


Figure 3.6: Lap Records by Season and Source

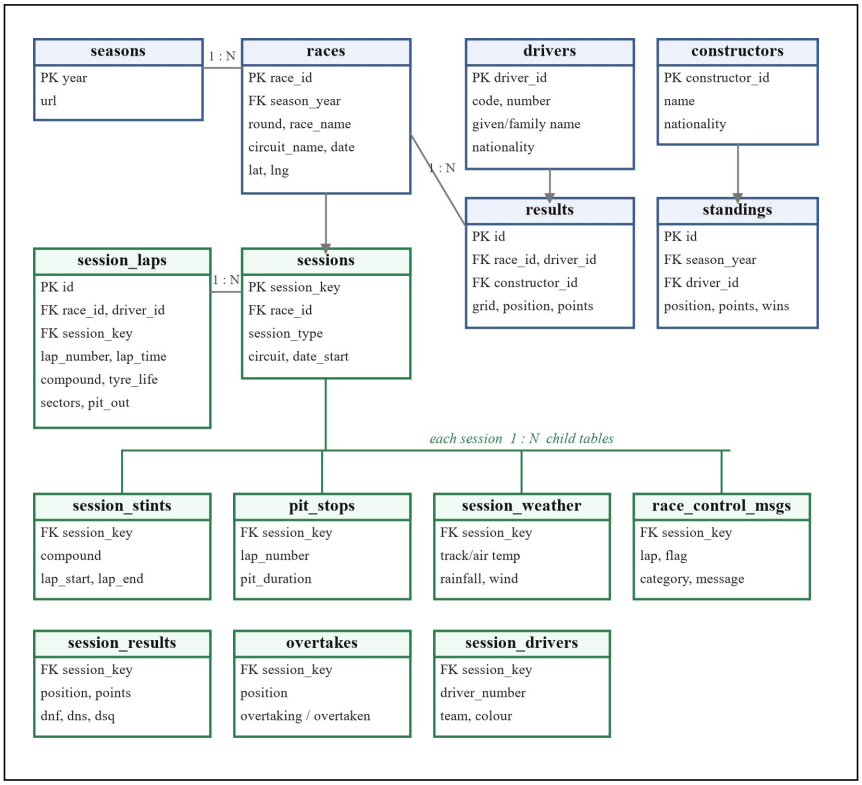


Figure 3.7: Entity-Relationship Diagram of the Database

Table 3.3: Database Tables

Table	Purpose
seasons	One row per championship year
races	Calendar entry: round, name, circuit, date, location
drivers / constructors	Driver and team master data
results / standings	Race classification and championship standings
sessions	Race and sprint sessions linked to their race
session_laps	Lap time, sectors, compound, tyre life, position
session_stints	Compound and lap range of each stint
pit_stops	Pit lap and stationary / lane duration
session_weather	Air and track temperature, humidity, rain, wind
race_control_messages	Flags, safety-car and other official messages
session_results	Session classification including DNF / DNS / DSQ
overtakes	Reconstructed position changes between cars
session_drivers	Driver number, acronym and team colour per session

To keep the data current without manual work, a background sync thread runs inside the backend. Its logic is shown in Figure 3.8. It checks frequently around race weekends and rarely otherwise, waits for the embargo period after each session, catalogues new sessions and harvests only the data that is not already stored.

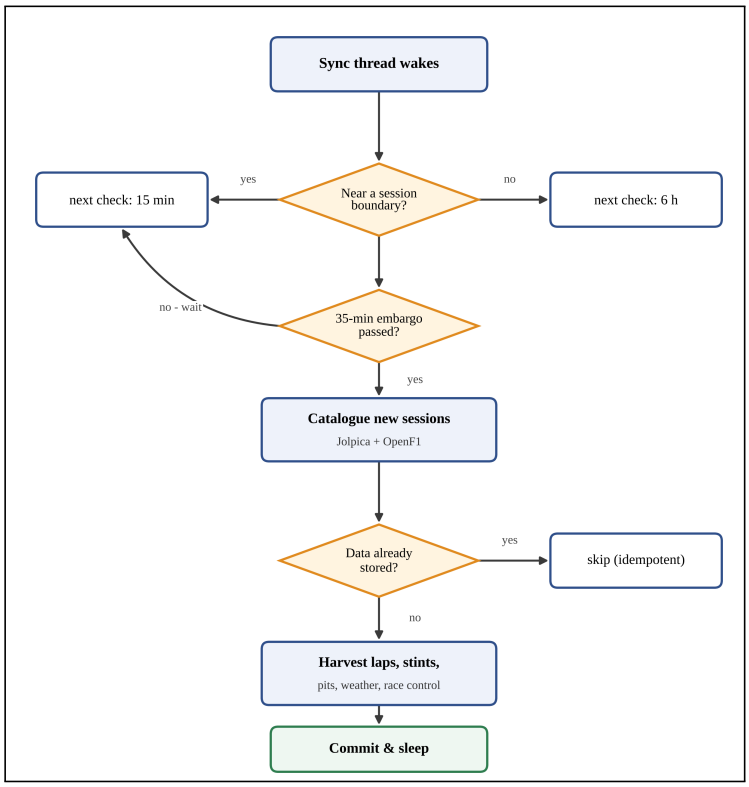


Figure 3.8: Background Data-Sync Flowchart

Table 3.4: Data Sources and Coverage

Source	Seasons	Contributes
Jolpica-F1 / Ergast	2002 - 2026	Results, grids, standings, calendar
OpenF1	2023 - 2026	Laps, stints, pit stops, weather, race control, car telemetry
FastF1	2018 - 2022	Lap backfill and tyre compounds

3.2.4 Phase 4: Model development

This phase defined which laps count as representative racing pace, which features describe a lap, what the model should predict and how its accuracy should be measured. Whole races are assigned to train, validation and holdout sets in a 70/15/15 seeded split of 58, 12 and 12 races, covering 40,012 clean laps from 2023 to 2026. Early stopping monitors the validation races only, and the holdout is scored exactly once. Sprint sessions are excluded because their lighter fuel load would teach a fuel effect that describes neither format. The details are given in Section 3.7.

3.2.5 Phase 5: Implementation

The backend was implemented in Python with FastAPI and SQLAlchemy and exposes 36 REST endpoints. The mobile client was implemented with Expo SDK 57 and React Native 0.86, using expo-router for navigation, TanStack Query for server-state caching and react-native-svg for charts, track maps and telemetry traces. A background daemon thread keeps the dataset fresh: it re-checks every 15 minutes near a session boundary and every 6 hours otherwise, respecting a 35-minute data embargo after each session.

3.2.6 Phase 6: Testing and deployment

The system underwent several levels of testing. Endpoints were verified individually and through the app, the background sync was run repeatedly to confirm it is idempotent, and the migration from SQLite to managed PostgreSQL copied all 15 tables with zero row-count mismatches. The backend is containerised with the dataset baked into the image and deployed on Render, the managed database runs on Supabase PostgreSQL, and the Android app is built with Expo EAS.

The resulting deployment is shown in Figure 3.9. The phone talks only to the backend container over HTTPS. The container holds the API, the model artifact, the sync thread and a baked-in SQLite snapshot, and it reaches the managed PostgreSQL database and the public Formula 1 APIs on the server side.

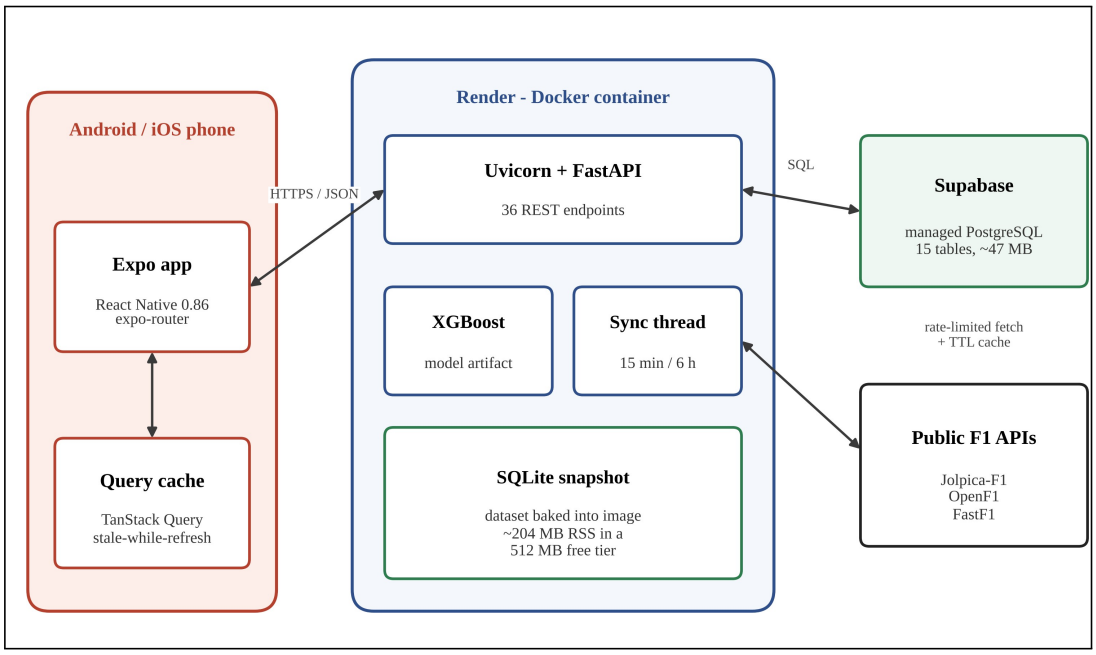


Figure 3.9: Deployment Diagram

Table 3.5: Types of Testing

Type	Description
Unit Testing	Lap filters, feature builder and simulator steps tested in isolation
Integration Testing	API endpoints exercised end to end through the app
Data Validation	Row counts verified against each source and after migration
Model Evaluation	Whole-race holdout scored once against two naive baselines
Performance Testing	Memory footprint and response time of the deployed container
Usability Testing	Navigation, readability and responsiveness of the mobile UI

3.3 Advantages of the Methodology

The adopted methodology provides a structured yet flexible approach. Fixing the data layer and the evaluation protocol early meant that every later improvement to the model or the interface could be measured against the same honest baseline. Sharing one clean-lap filter across training, analytics and the API removed a whole class of inconsistencies. Combining a relational store with cached on-demand telemetry kept the database small and the hosting free. Overall, the methodology produced a system that is reproducible, cheap to run and explicit about its own uncertainty.

3.4 System Workflow

The central workflow of OpenTyreF1 is the strategy-simulation request shown in Figure 3.10. The user selects a race, a driver and a lap; the backend loads that race's laps, stints, pit stops and weather and passes them through the clean-lap filter. The user then builds a stint plan. The XGBoost model predicts every lap of the plan, laps run behind a safety car are clamped to the field's observed pace, and a pit loss measured at that circuit is charged for each stop. The resulting cumulative time is ranked against every rival's real cumulative time, a Monte Carlo pass estimates the spread of finishing positions, and the app shows the result with a fact-backed explanation.

The application is organised into four tabs and several detail screens:

- **Live Tab:** The entry point of the app. It shows the latest result or the next race with a countdown, the timing tower with podium, fastest lap and biggest gainer, the winning strategy and conditions, and a Track view with the circuit map, live conditions and recent winners.
- **Drivers Tab:** A searchable list of drivers with team filters. Each driver screen shows career wins, podiums, poles and titles, a season-by-season breakdown, race-by-race results and full team history.
- **Race Detail Screens:** Race replay with lap-by-lap position playback, pace and position charts, two-driver telemetry comparison (speed, throttle, brake and DRS) and an analysis view with fuel-corrected tyre degradation and teammate gaps.
- **Strategy Tab:** A module picker for the stint planner and the seven pit-wall modules. The stint planner lets the user build a plan of compounds and pit laps and run it against the driver's real race.
- **Assistant Tab:** A grounded question-answering assistant. Every answer is built from the database and the simulator and lists the facts it used.

The navigation structure that links these screens is shown in Figure 3.11. Each tab opens its own stack of screens, so a user can move from the latest result to a full race analysis, or from the strategy picker to a simulation result, without losing their place.

3.5 Tools and Technologies Used

3.6 System Architecture

OpenTyreF1 follows a modular, layered client-server architecture designed for reproducibility, low running cost and fast response on a mobile network. At a high level it consists of the layers shown in Figure 3.12, each responsible for a specific set of func-



Figure 3.10: System Workflow Diagram

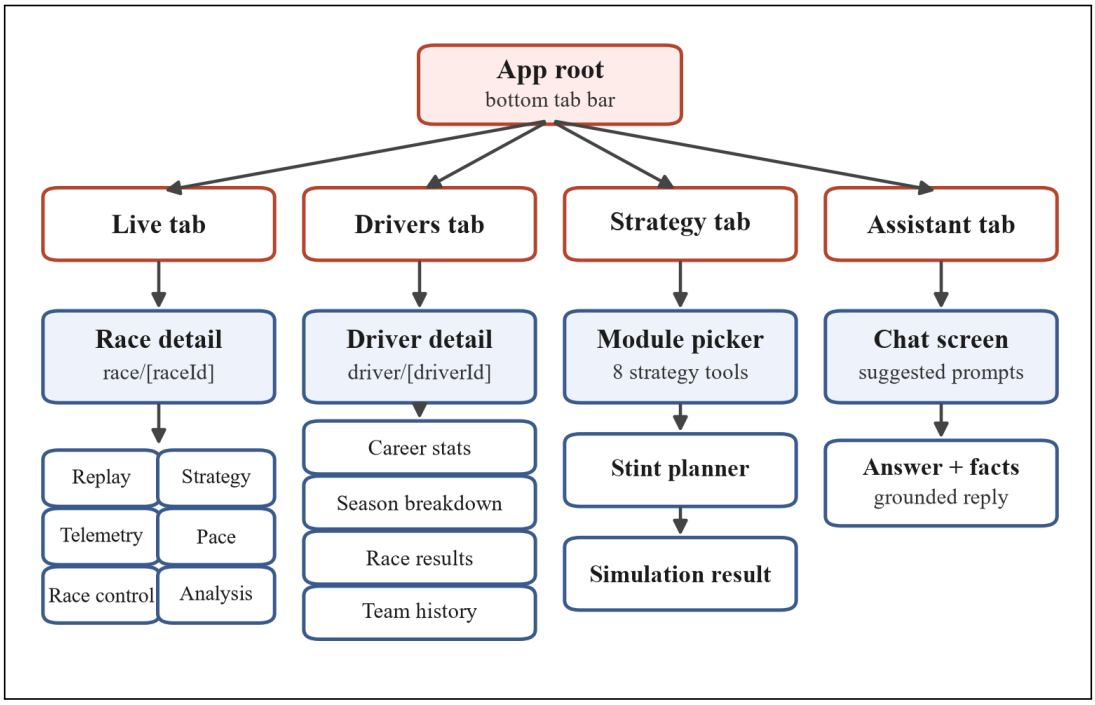


Figure 3.11: Application Navigation Map

Table 3.6: Tools and Technologies

Components	Technologies / Tools	Purpose
Mobile Frontend	Expo SDK 57, React Native 0.86, TypeScript, expo-router	Cross-platform app with tab and stack navigation
UI and Charts	react-native-svg, TanStack Query	Charts, track maps, telemetry traces and cached server state
Backend	Python 3.11, FastAPI, Uvicorn, Pydantic	36 REST endpoints with an OpenAPI contract
Database	SQLAlchemy, SQLite, PostgreSQL (Supabase)	Portable 15-table relational store
Machine Learning	XGBoost, scikit-learn, pandas, NumPy	Lap-time model, evaluation and analytics
Data Sources	Jolpica-F1, OpenF1, FastF1	Historical results, live timing and lap backfill
Deployment	Docker, Render, Expo EAS, Git / GitHub	Container hosting, app builds and version control

tions.

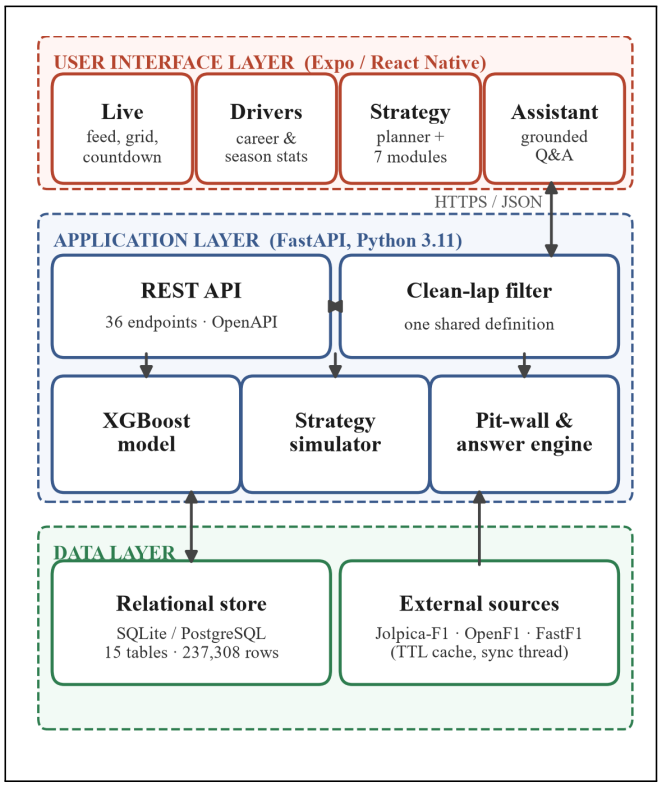


Figure 3.12: System Architecture Diagram

3.6.1 Presentation layer (mobile app)

This layer is the Expo / React Native application with four tabs (Live, Drivers, Strategy and Assistant) plus race, driver and strategy detail screens. Server state is held in a TanStack Query cache, so screens render instantly from cache and refresh in the background. Charts, track maps and telemetry traces are drawn with react-native-svg. A dark, pit-wall inspired theme keeps dense data readable on a small screen.

3.6.2 Application layer (backend)

The backend is a FastAPI service exposing 36 REST endpoints grouped into routers for health, drivers, races, sessions, telemetry, strategy, live data and the assistant. It handles request validation, pagination and error handling, and it calls into the service layer, which hosts the clean-lap filter, analytics, the simulator, the pit-wall modules and the answer engine.

Table 3.7: Main REST API Endpoints

Method	Endpoint	Purpose
GET	/api/health	Service and database health check
GET	/api/live/feed	Latest result or next race with countdown
GET	/api/drivers/{driver_id}	Driver career statistics and history
GET	/api/races/seasons	List of available seasons
GET	/api/races/{race_id}/replay	Lap-by-lap positions for race replay
GET	/api/races/{race_id}/analysis	Pace, degradation and teammate gaps
GET	/api/telemetry/compare	Two-driver speed, throttle, brake, DRS
GET	/api/sessions/{session_key}/track	Circuit outline from telemetry coordinates
GET	/api/strategy/options/{race_id}	Drivers, race distance and pit loss
POST	/api/strategy/simulate	Run a stint plan against the real field
GET	/api/strategy/safety-car/{race_id}	Pit-wall module (one of seven)
POST	/api/ai/query	Grounded assistant question answering

3.6.3 Machine learning and simulation layer

This is the analytical core of the system. It contains the dataset builder, the XGBoost trainer and predictor, the counterfactual strategy simulator, the Monte Carlo outcome estimator and the seven pit-wall decision modules. The trained model is stored as an artifact together with its metrics, and every simulation response carries a modelSource field stating which path produced the numbers.

3.6.4 Data layer (database)

The data layer is a 15-table relational store, portable between SQLite (shipped inside the repository, about 27 MB) and managed PostgreSQL through one SQLAlchemy layer. Because the database ships with the code, a fresh clone runs immediately without any ingestion step. A CSV export of every table is also provided for inspection.

3.6.5 Integration layer (external services)

The integration layer connects the backend to Jolpica-F1, OpenF1 and FastF1. Car telemetry and track coordinates are fetched on demand through a time-to-live (TTL) cache instead of being bulk-stored. Funnelling all upstream traffic through this cache ensures that OpenF1's free-tier limit of about three requests per second is never exceeded, no matter how many users open the app.

3.7 Core Algorithms and Modules

3.7.1 Clean-lap filter

The clean-lap filter decides which laps count as representative racing pace. It is the single highest-leverage component, because it defines the training data, every published pace statistic and every degradation estimate. It is one function imported by the trainer, the analytics services and the API, so they can never disagree. Laps are admitted by four successive exclusions:

1. **Opening lap:** a standing start is not a measure of racing pace.
2. **Pit-affected laps:** out-laps are flagged in the source; in-laps are recovered from the pit-stop table.
3. **Neutralised laps:** safety-car and virtual-safety-car periods are reconstructed from free-text race-control messages with a single-pass state machine.
4. **Per-driver 107% rule:** laps slower than 1.07 times the driver's own clean median are removed. A per-driver threshold avoids over-pruning slower cars.

Neutralised periods are not given as a column in the source data; they must be recon-

structed from the free-text race-control messages. The state machine used for this is shown in Figure 3.13. Every lap that starts or ends while the track is in the SC, VSC or red-flag state is marked as neutralised.

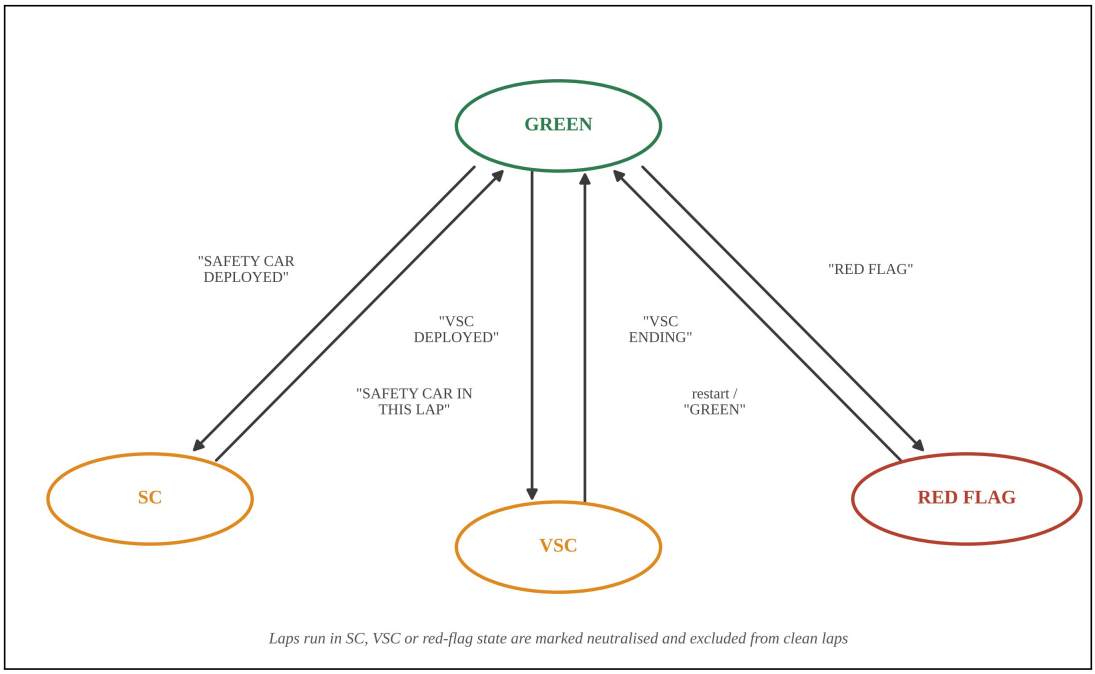


Figure 3.13: Track-Status State Machine

Over 93 ingested race sessions, 58,848 of 93,282 timed laps survive, a retention of 63.1%, as shown in Figure 3.14. Figure 3.15 shows the effect on a single race: the long tail of slow laps caused by pit stops and safety cars disappears, leaving a compact distribution that reflects true racing pace.

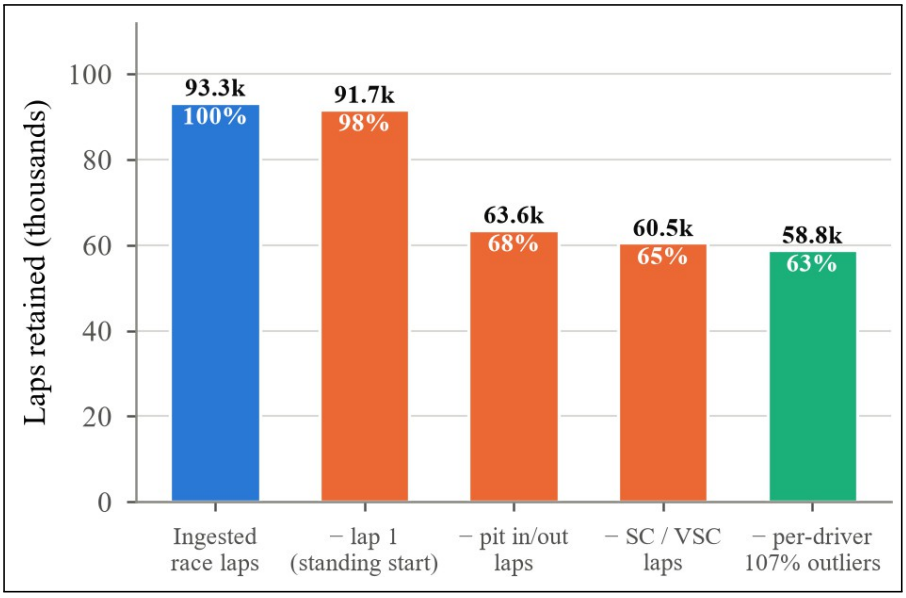


Figure 3.14: Clean-Lap Admission Cascade

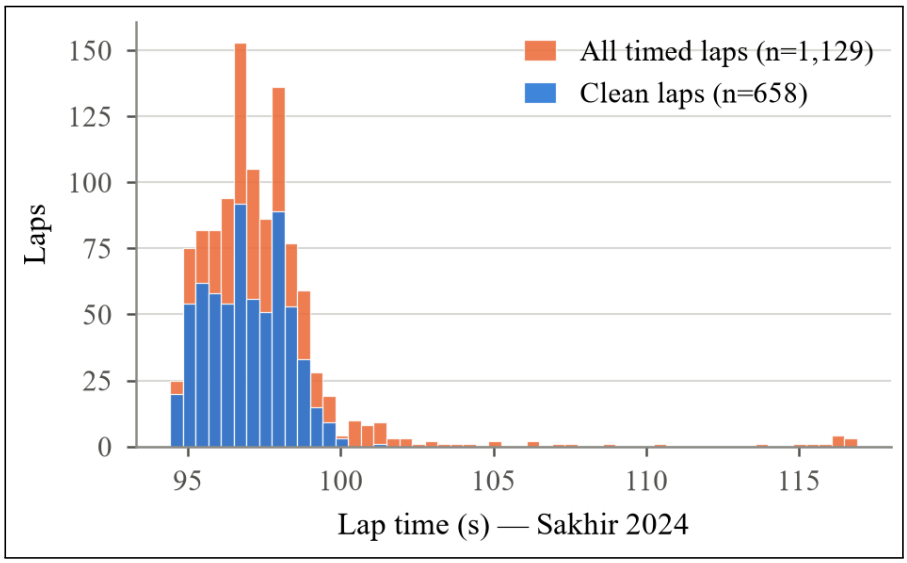


Figure 3.15: Lap-Time Distribution Before and After Filtering

3.7.2 Lap-time prediction model

The model predicts the time of a lap from six features: tyre compound, circuit and driver (categorical), and tyre age, lap number and track temperature (numeric). Weather is joined as of each lap's start: the lap takes the most recent reading at or before it begins, found by binary search. A lap is never described by a measurement taken after it finished, and the same code path is used in training and in simulation.

Rather than regressing the absolute lap time y, the model predicts the offset from the race's median clean lap m, which is added back at scoring time:

$$\hat{y} = m + f(\mathbf{x}), \qquad \text{where } f \text{ is trained on } (y - m) \tag{3.1}$$

This is legitimate for the simulator, which always operates on a completed historical race whose median is known at prediction time, exactly as in training. The data and training pipeline is summarised in Figure 3.16 and the model configuration in Table 3.8.

Table 3.8: XGBoost Configuration

Parameter	Value	Parameter	Value
n_estimators	1200	subsample	0.85
learning_rate	0.05	colsample_bytree	0.85
max_depth	6	reg_lambda	1.5
min_child_weight	4	early stopping rounds	22

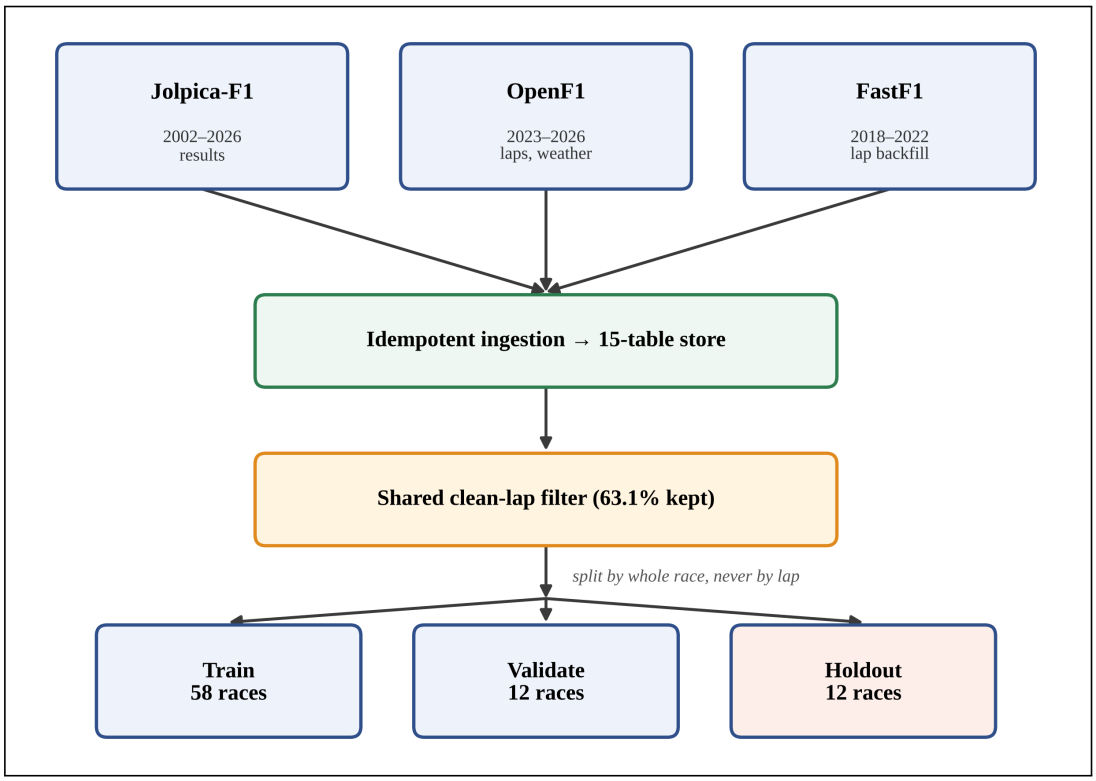


Figure 3.16: Data and Model-Training Pipeline

3.7.3 Strategy simulation module

Given a real race, a real driver and a hypothetical stint plan, the simulator estimates where that driver would have finished. It performs five steps:

1. Expand the plan into a per-lap feature sequence over the real race distance.
2. Predict each lap with the trained model.
3. Clamp neutralised laps: if the whole field ran more than 10% slower than reference pace on a lap, the simulated car follows the field's observed time.
4. Charge pit losses measured at that circuit: observed stop durations are clamped to 15 to 45 seconds and the median is taken. With fewer than five samples, a declared 22-second default is used and reported.
5. Rank the counterfactual cumulative time against every rival's real cumulative time, lap by lap, to produce a finishing position.

The interaction between the components during a simulation request is shown in the sequence diagram of Figure 3.17. The app sends one request; the backend resolves the session and weather, loads laps, stints and pit samples, builds the per-lap feature matrix, obtains predictions from the model, applies the safety-car clamp and pit losses, and returns frames, deltas and the final position.

If no trained model is available, the simulator falls back to an empirical linear degrada-

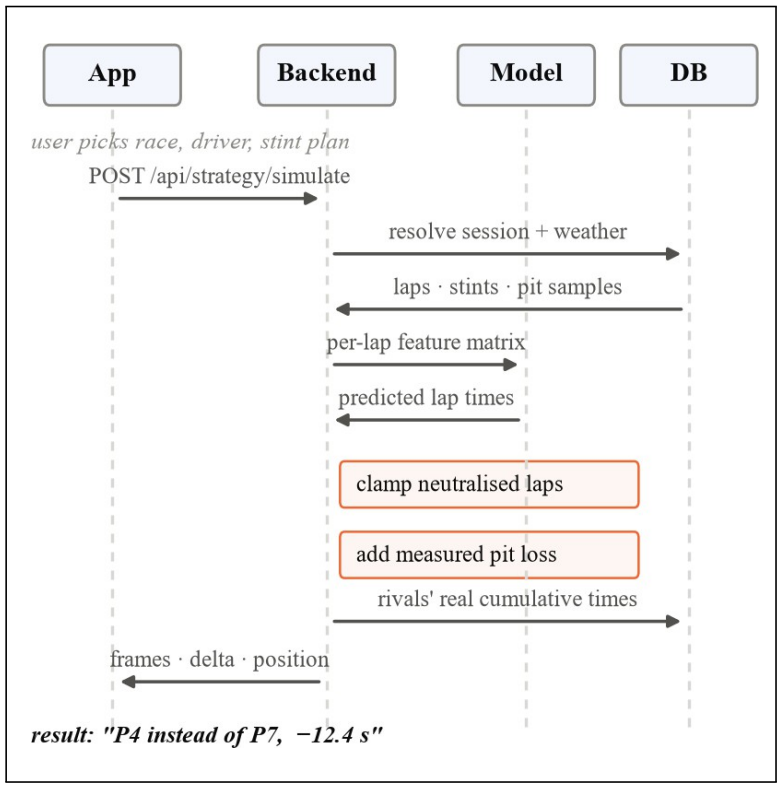


Figure 3.17: Sequence Diagram of a Strategy Simulation

tion fit with declared per-compound rates of 0.085, 0.050 and 0.030 seconds per lap for soft, medium and hard tyres.

3.7.4 Monte Carlo outcome module

A single simulated position conveys no uncertainty, so the outcome module runs 1,500 bootstrap replications [15] from a chosen lap. Laps already completed are held fixed. Only the remainder of the race is resampled from the driver's clean laps, perturbed by the model's published error, charged once for each remaining pit stop and exposed to historical safety-car and retirement rates per lap. The output is a probability distribution over finishing positions.

Resampling only the remainder of the race is what makes the estimate useful during a race replay. As the chosen lap moves later, more of the race is held fixed and the distribution narrows, so the user can see how certainty about the result grows lap by lap. Drawing from the driver's own clean laps, rather than from a generic distribution, keeps every replication consistent with that driver's real pace at that circuit.

The seven steps of each Monte Carlo replication are shown in Figure 3.18. Repeating them 1,500 times gives a stable estimate of the probability of each finishing position, from which the position-prediction and risk-analysis modules derive their outputs.

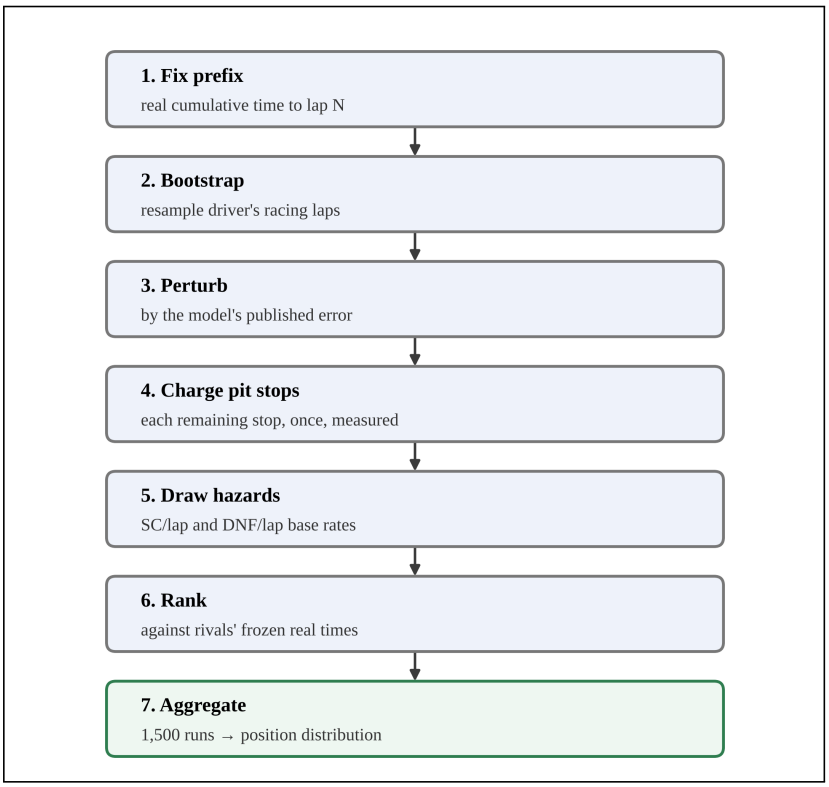


Figure 3.18: Monte Carlo Outcome Procedure

3.7.5 Pit-wall decision modules

Seven modules answer bounded in-race questions from historical base rates (Table 3.9). All share one response format: a verdict, a confidence, an expected gain, the reasoning, the supporting facts and the sample size behind them. A recommendation derived from 13 observations is therefore never presented with the same authority as one backed by thousands.

Table 3.9: Pit-Wall Strategy Modules

Module	Basis
Safety Car	Pit vs. stay-out outcomes under SC / VSC periods
Red / Yellow Flag	Reconstructed flag periods and their outcomes
Weather	Intermediate / wet crossover base rates
Overtake	Pass rate by gap bucket (< 0.5 s to > 1.5 s)
Defence	The same position-swap data, from the defender’s side
Position Prediction	Monte Carlo finishing-position distribution
Risk Analysis	Downside summary of the same Monte Carlo runs

3.7.6 Grounded assistant module

The assistant is a retrieval-and-template engine, not a generative language model. A question is scored against keyword sets for each intent, entities such as drivers and races are resolved from the text or the app's context, and a per-intent renderer turns database rows into prose together with the list of facts used. Because it can only state what retrieval returned, it cannot produce a fluent claim that no data supports. An unanswerable question returns an explicit statement and the list of questions it can answer.

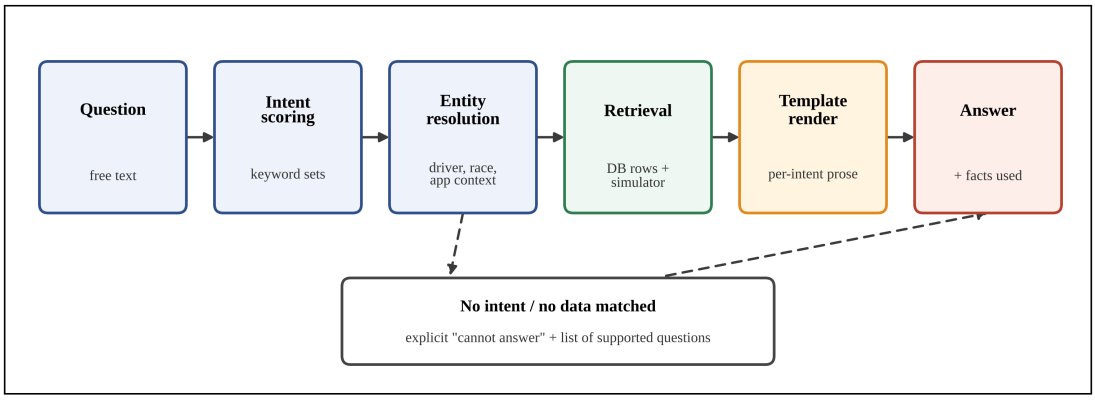


Figure 3.19: Grounded Assistant Pipeline

The assistant currently recognises intents such as explaining a simulation, finding the best strategy at a race, summarising a race, reporting degradation, comparing two drivers, giving driver statistics and explaining the methodology of the model itself. Each answer is returned with the list of facts and sources it used, which the app displays under the reply.

3.8 Error Handling and Data Integrity

Because the system depends on several external services and on data that is sometimes incomplete, robustness was treated as a design requirement rather than an afterthought. The main measures are:

- **Input validation:** request bodies are validated with Pydantic models. An unknown tyre compound is rejected with HTTP 400 or 422 and an unknown driver with HTTP 404, instead of the backend guessing what the user meant.
- **No fabricated records:** a session that cannot be matched to a race is skipped with a logged reason rather than stored under an invented parent row.
- **Idempotent ingestion:** every ingestion routine checks what is already stored before calling an upstream API, so an interrupted run can simply be restarted.
- **Rate-limit protection:** all third-party traffic passes through the backend's TTL cache, and the catalogue and harvest steps of the sync are separated so that a sync pass never bursts above the free-tier limit.

- **Declared fallbacks:** when a circuit has fewer than five pit-stop samples or no trained model is available, the system uses a declared default and says so in the response.
- **Safe configuration:** database credentials are supplied through environment variables, and the files that hold them are excluded from version control.
- **Graceful client behaviour:** the app shows cached data while a request is in flight and displays a clear message when the backend or an upstream source is unavailable.

Together, these measures mean that the system fails loudly rather than silently. When data is missing or a request cannot be answered, the user is told so directly, and the response states which fallback, if any, produced the numbers. This behaviour was verified with the end-to-end test script described in Chapter 5, which exercises unknown drivers, unknown compounds and unanswerable questions against the real database.

STRENGTH & LIMITATION

Every system is designed with certain capabilities that enhance its performance, while also having some limitations that may affect it in specific scenarios. Understanding both is essential for evaluating the overall effectiveness of OpenTyreF1. The strengths highlight its accuracy, transparency and practicality, whereas the limitations identify areas that require improvement and give direction for future work.

The strengths and limitations below were identified by testing the complete system against real races rather than synthetic data, so each one is tied to a measured behaviour of the deployed application.

4.1 Strengths

OpenTyreF1 provides several advantages that make it useful for both technical and non-technical users.

- **Honestly Evaluated Model:** The lap-time model is scored on 12 whole races it has never seen and beats the strongest naive baseline by 0.270 seconds of mean absolute error, so the reported accuracy reflects real-world use.
- **Counterfactual Strategy Analysis:** Unlike descriptive tools, the system answers "what if" questions by running a hypothetical stint plan against the real field.
- **Explicit Uncertainty:** Monte Carlo simulation and sample-size reporting tell the user how much to trust each answer.
- **Grounded Assistant:** Every answer is built from database rows and simulator output and lists the facts used, so the assistant cannot invent numbers.
- **Single Source of Truth:** One shared clean-lap filter keeps the model, the analytics and the app consistent.
- **Rich Unified Dataset:** 237,308 records covering 492 races from 2002 to 2026 are reconciled from three public sources into one normalised schema.
- **Low Cost and Reproducibility:** The database ships with the code, the backend fits a 512 MB free-tier host, and ingestion is idempotent and rate-limit safe.
- **Cross-Platform Mobile Experience:** A single Expo / React Native codebase serves Android and iOS with cached, responsive screens.

4.2 Limitations

- **Pit-Lap Rule:** In-laps are excluded by lap number for every driver instead of per (driver, lap) pair. This discards 30.7% of candidate laps where 6.2% would suffice. The bias is conservative, but it costs about 22,000 usable laps.
- **Single-Agent Simulation:** Rivals are frozen at their real lap times, so the simulator does not model how rivals would react to a different strategy.
- **Retrospective Target:** The relative target needs the race median, so live in-race prediction would require an online estimate of it.
- **Coverage:** The model is trained on 2023 to 2026 races only; the 2018 to 2022 backfilled laps are stored but not yet usable for replay.
- **Sample Size:** A 12-race holdout cannot resolve differences below about 0.05 seconds of MAE.
- **Dependency on Public APIs:** Live data depends on the availability and rate limits of community-run services such as OpenF1.
- **No Car-Setup or Team-Radio Data:** Public sources do not expose car setup, tyre pressures or team communications, which real strategists use.

None of these limitations invalidates the reported results, because each one is either conservative, removing data rather than admitting bad data, or explicitly declared in the system's responses. They do, however, bound how the results should be used: the simulator answers what would have happened if one driver alone had chosen differently, not how the whole race would have unfolded, and its accuracy figures apply to races from 2023 onwards.

4.3 Use Cases for Software

OpenTyreF1 can be applied in several real-world scenarios across different domains.

- **Formula 1 Fans:** Fans can replay races, compare drivers' telemetry and test whether a different pit strategy would have changed the result.
- **Students and Educators:** The system is a complete, real-world example of data engineering, machine learning, simulation and mobile development, suitable for teaching and academic projects.
- **Motorsport Analysts and Journalists:** Analysts can back their commentary with measured tyre degradation, pit-loss figures and simulated outcomes rather than intuition.
- **Junior and Amateur Racing Teams:** The same methodology can be applied to lap data from lower categories to plan strategies at low cost.

- **Fantasy and Prediction Platforms:** Position distributions and historical base rates can inform fantasy-league and prediction-game decisions.
- **Data-Science Research:** The reproducible corpus and leakage-free protocol provide a benchmark for research on lap-time prediction and strategy optimisation.
- **Broadcast and Content Creation:** Commentators and content creators can illustrate strategy stories with simulated alternatives and measured degradation instead of hand-drawn sketches.

Across all these use cases, the common value of OpenTyreF1 is that it turns open data into answers that can be checked. Users do not have to trust a black box: they can see the model source, the sample size and the facts behind every number, which makes the system suitable both for casual exploration and for serious analysis.

RESULT & DISCUSSION

OpenTyreF1 successfully demonstrates that open Formula 1 data can support honest, counterfactual strategy analysis on a mobile phone. The lap-time model achieves strong accuracy on unseen races, the simulator turns predictions into finishing positions, and the application presents the results clearly. This chapter reports the measured performance of the system and discusses what the results mean.

5.1 System Performance

The production container uses about 204 MB of resident memory with the model loaded, which fits comfortably in a 512 MB free-tier host. The serving image omits FastF1 and its scientific stack, so no ingestion code can run on the request path and container builds take seconds instead of minutes. A full background sync pass completes in seconds, is idempotent across repeated runs and caused no rate-limit hits after the catalogue and harvest steps were separated. The migration to managed PostgreSQL copied all 15 tables with zero row-count mismatches, using about 47 MB of storage. On the client, screens render from the query cache immediately and refresh in the background.

5.2 Functional Testing

Functional testing was carried out with an end-to-end verification script that drives the real API against the real ingested database. This is deliberate: the failures that matter in this system, such as a race with no pit data, a driver who retired or a model artifact that does not match the features the simulator sends, only appear with real data. The script exits with an error if any check fails, so it can gate a demonstration. Table 5.1 lists representative test cases.

5.3 Prediction Accuracy

Table 5.2 and Figure 5.1 report holdout performance on the 12 unseen races. The model attains a mean absolute error of 0.893 s and $R^2 = 0.978$, improving on the matched baseline B2 (the race's own median clean lap) by 0.270 s of MAE. Baseline B1, which knows circuit and driver identity but nothing about within-race evolution, still reaches $R^2 = 0.545$. This confirms that circuit identity alone explains much of the pooled variance, which is why a meaningful baseline must already encode it. Accuracy is computed as:

Table 5.1: Representative Test Cases

ID	Test Case	Expected Result
TC-01	GET /api/health	HTTP 200 with database status
TC-02	Model metadata	Model beats its baseline; holdout and validation are whole races
TC-03	Strategy options for a race	Drivers, race distance and a plausible pit loss are returned
TC-04	Simulate the real plan	Total time close to the driver's real race time
TC-05	Simulation frames	Full field each frame, positions 1..n without gaps, leader gap zero
TC-06	Cumulative time	Increases monotonically lap by lap
TC-07	Different strategy	A one-stop gives a different total; fewer stops, less pit time
TC-08	Unknown compound	Rejected with HTTP 400 / 422, not guessed
TC-09	Unknown driver	Rejected with HTTP 404
TC-10	Assistant intents	Correct intent for simulation, race, degradation and comparison questions
TC-11	Ungrounded question	Refused, with a list of questions it can answer
TC-12	Answer sources	Every grounded answer cites its sources

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2} \tag{5.1}$$

Table 5.2: Holdout Performance (12 Unseen Races)

Predictor	MAE (s)	RMSE (s)	R^2
XGBoost (proposed)	0.893	1.205	0.978
B2: race median lap	1.163	1.494	0.965
B1: driver × circuit median	4.096	5.420	0.545

5.4 Target Design versus Feature Richness

Figure 5.2 crosses three feature sets with both targets. With the feature set fixed, switching from the absolute to the relative target cuts MAE from 3.586 s to 0.893 s, a factor of four. With the target fixed, almost doubling the feature count from 6 to 11 improves MAE by only 0.033 s. That is inside the declared parsimony tolerance of 0.05 s, so the simpler six-feature model is the one shipped. Early stopping chose iteration 16 of a permitted 1,200, which suggests that once the target is well parameterised the problem is close to linear in these features.

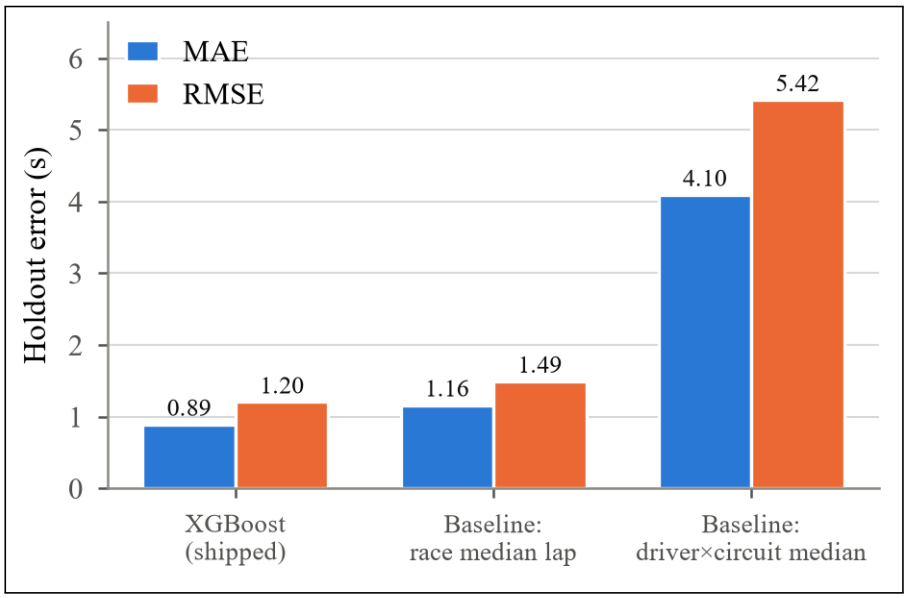


Figure 5.1: Holdout Error of the Model against Both Baselines

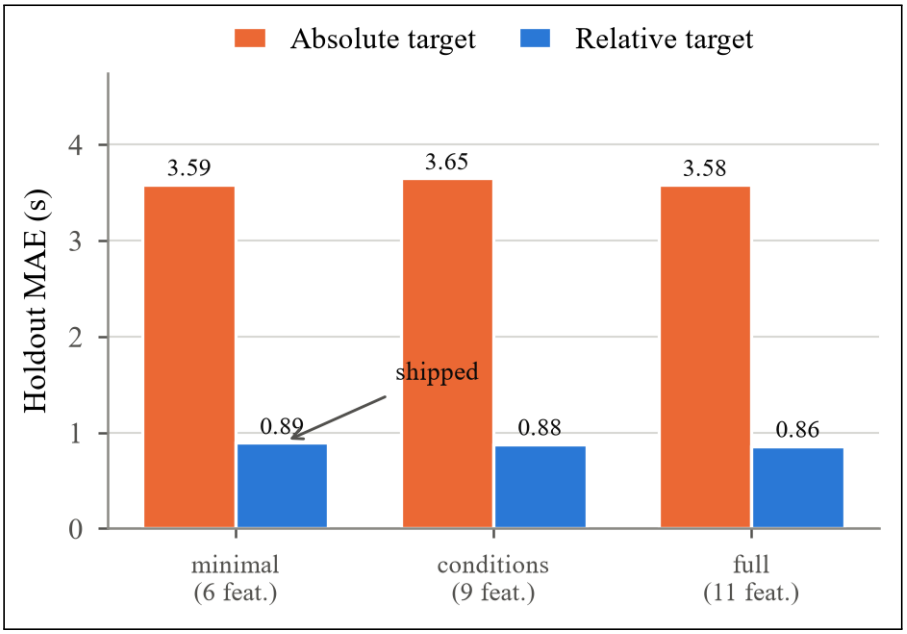


Figure 5.2: Ablation - Target Parameterisation vs. Feature Set

Gain importance is led by lap number (0.422), which acts largely as a fuel-mass proxy, followed by driver (0.185), circuit (0.122), track temperature (0.102), compound (0.091) and tyre age (0.078). Tyre age ranks low because it advances in lockstep with lap number within a stint, and gain attribution splits credit between correlated features arbitrarily. It should not be read as tyres being unimportant, as the next result shows.

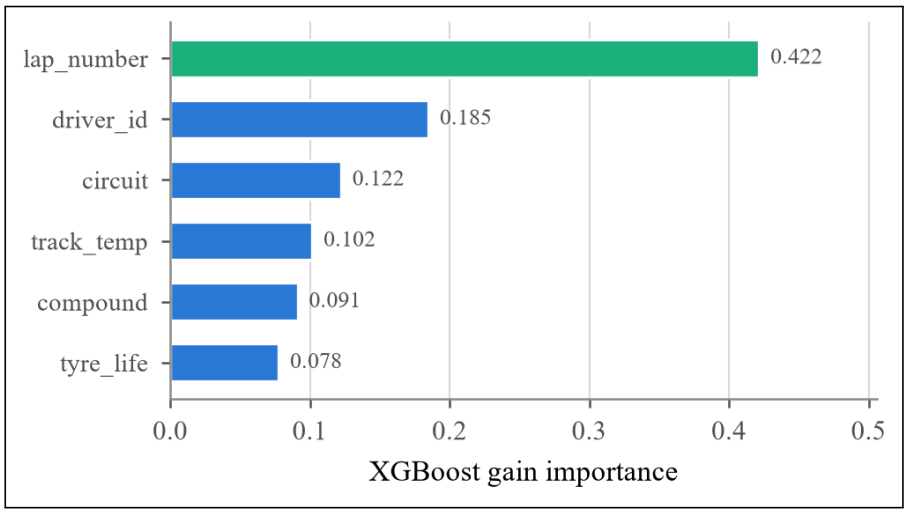


Figure 5.3: XGBoost Gain Importance of the Six Features

5.5 Fuel-Corrected Tyre Degradation

Figure 5.4 shows how lap records are spread across tyre compounds. Medium and hard tyres dominate race running, soft tyres account for most of the rest, and the 2018 compound names (supersoft, ultrasoft, hypersoft) appear only in the backfilled seasons. Wet-weather compounds are rare, which limits what can be learned about them.

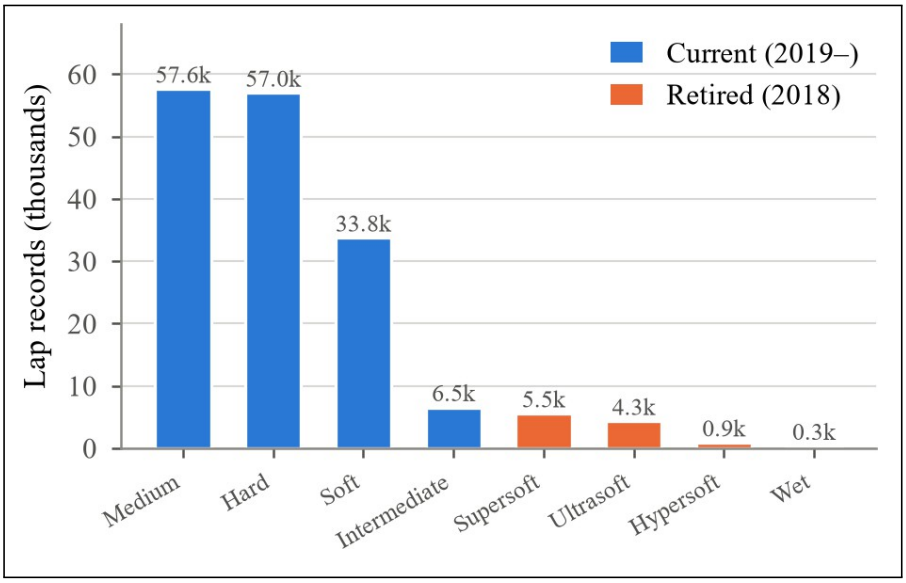


Figure 5.4: Lap Records by Tyre Compound

Raw lap-time-versus-tyre-age slopes mix tyre wear with fuel burn. A per-session fuel slope, fitted by least squares on lap number, is therefore subtracted from each driver-stint's tyre-age slope. Aggregated over the corpus (Figure 5.5), soft tyres degrade fastest at a median 0.057 s per lap, against 0.037 s for hard and 0.034 s for medium. Medium and hard are statistically indistinguishable here. Intermediates show -0.114 s per lap: a drying track gets faster faster than the tyre wears, which marks where the linear estimator stops being valid.

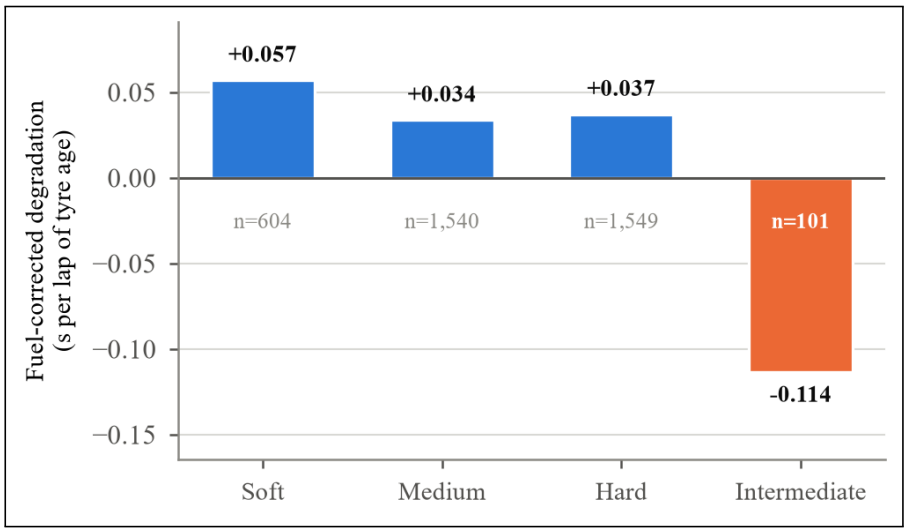


Figure 5.5: Median Fuel-Corrected Degradation by Compound

5.6 User Experience

The application is designed to make dense data easy to read. The four-tab layout keeps the main features one tap away, and the circuit map sits behind a segmented switch next to the timing tower instead of being buried below twenty rows. The stint planner uses a single bar for the whole race: tapping a block changes its compound, and pit laps are adjusted with simple plus and minus controls. Simulation results are presented as a projected finish, a time delta against the real race and a stint-by-stint breakdown, so users can see exactly where time was gained or lost. Because screens render from cache, navigation feels immediate even on a slow mobile connection.

5.7 Discussion

The central finding of this project is methodological. On data of this size and shape, the decisive choices are not about the learner but about which laps are admitted, how the target is parameterised and how evaluation is partitioned. Reparameterising the target improved MAE four-fold, while nearly doubling the feature count moved it by only 0.033 s. A random lap-level split would have reported a much better, and misleading, score; the whole-race holdout gives an estimate that users can rely on.

The system also shows that grounding and uncertainty can be built into a consumer product. Every simulation states which model produced it, every pit-wall module reports its sample size, and the assistant cites its facts. At the same time, the limitations identified in Chapter 4, particularly the conservative pit-lap rule and the single-agent simulator, mark clear and measurable directions for improvement.

5.8 Results

OpenTyreF1 met all of its objectives. Three public sources were reconciled into one 237,308-record corpus covering 492 races. The shared clean-lap filter retains 63.1% of timed laps. The XGBoost model reaches 0.893 s MAE and $R^2 = 0.978$ on unseen races. The simulator, the Monte Carlo estimator and seven pit-wall modules turn those predictions into decisions, and the mobile app delivers everything through a fast, responsive interface backed by a 204 MB container on free-tier hosting.

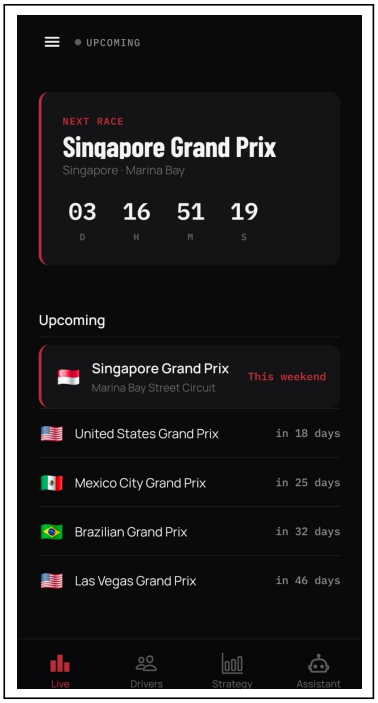
Table 5.3: Objectives and Achieved Outcomes

Objective	Outcome
Integrate public F1 sources into one corpus	237,308 records, 492 races (2002-2026), 15 tables, idempotent ingestion
One shared clean-lap filter	Single function used by trainer, analytics and API; 63.1% of timed laps retained
Leakage-free lap-time model	Whole-race 70/15/15 split; 0.893 s MAE, 1.205 s RMSE, R^2 0.978 on 12 unseen races
Counterfactual strategy simulator	Five-step simulator ranked against the real field, with declared fallbacks
Uncertainty and pit-wall decisions	1,500-run Monte Carlo and seven modules reporting verdict, confidence and sample size
Fast cross-platform app with assistant	Expo / React Native app with four tabs and a grounded assistant citing its facts

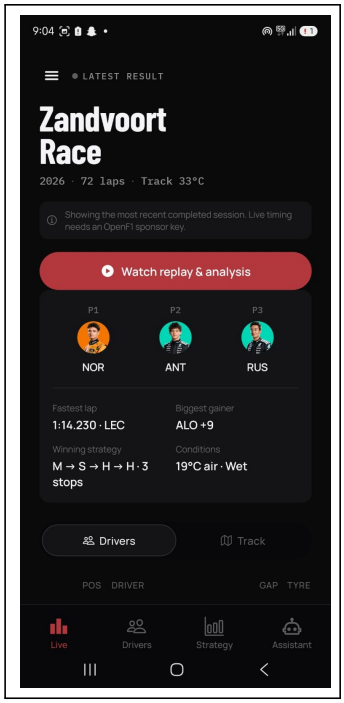
In summary, the results confirm that each objective set out in Chapter 1 has been met with measured evidence, and that the limitations that remain are known, quantified and documented.

5.9 Screenshots

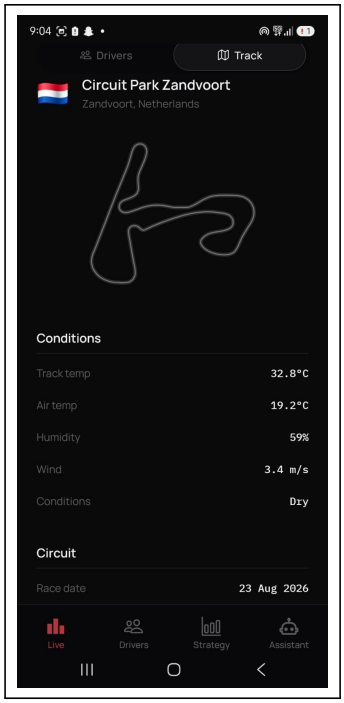
The screenshots below were captured from the Android build of the application running against the deployed backend. They follow the four tabs of the app: the Live tab with the next-race countdown, latest result and track conditions; the Drivers and race screens with replay, pace, telemetry comparison and tyre-degradation analysis; the Strategy tab with the module picker, stint planner and simulation result; and the grounded Assistant.



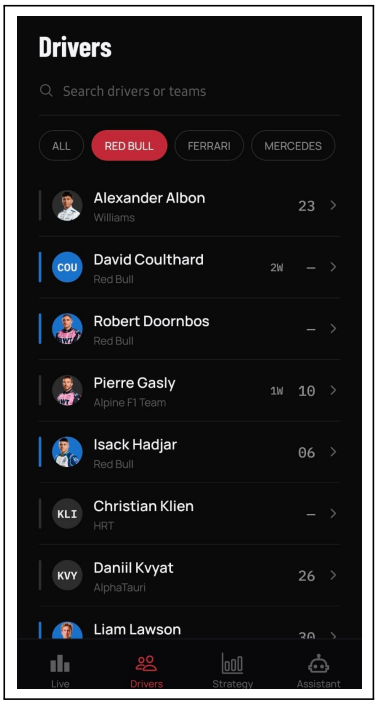
Screenshot 5.1: Live Tab - Next Race Countdown



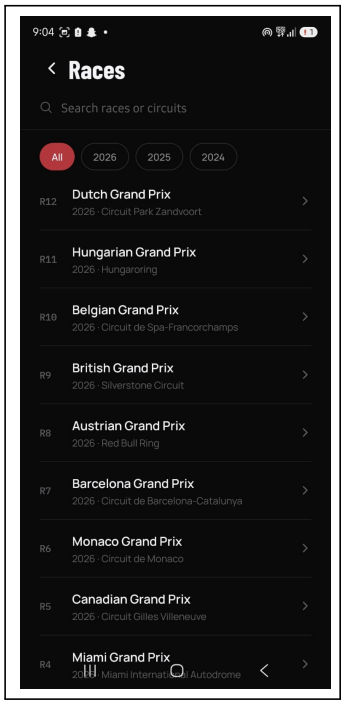
Screenshot 5.2: Latest Result and Timing Tower



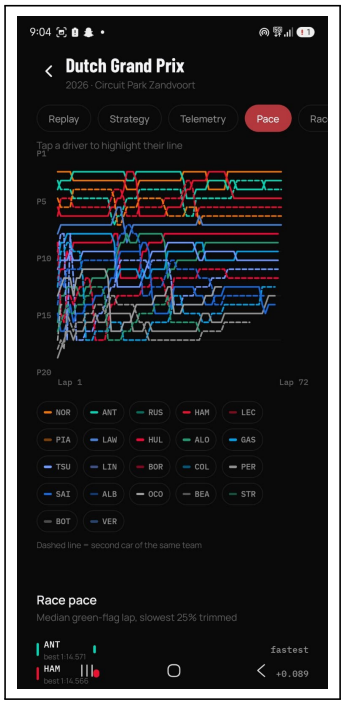
Screenshot 5.3: Track Map and Conditions



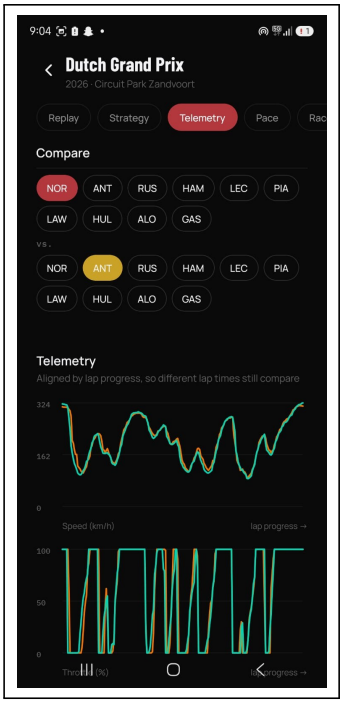
Screenshot 5.4: Drivers List



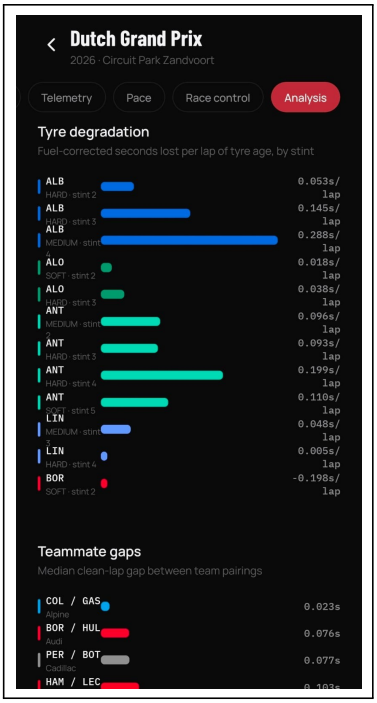
Screenshot 5.5: Races List



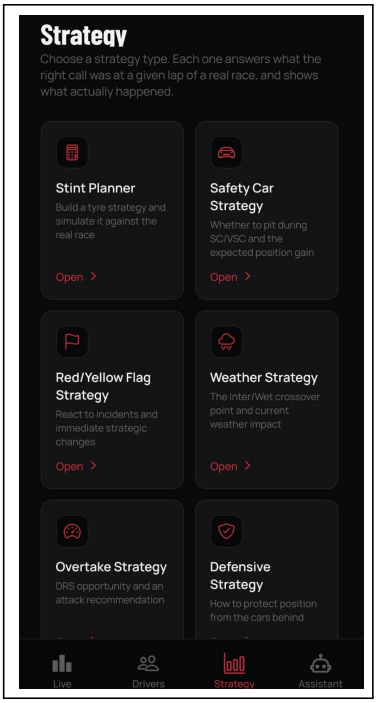
Screenshot 5.6: Race Pace and Positions



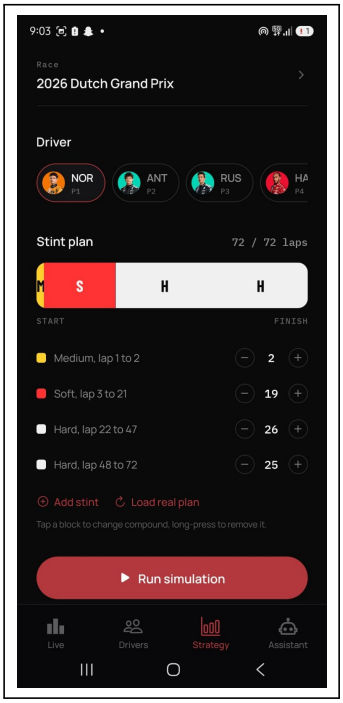
Screenshot 5.7: Telemetry Comparison



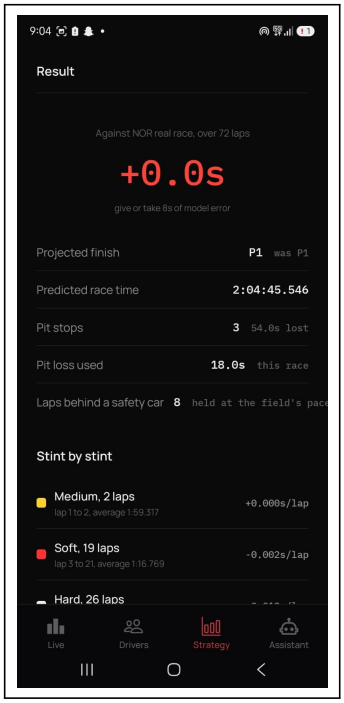
Screenshot 5.8: Tyre Degradation Analysis



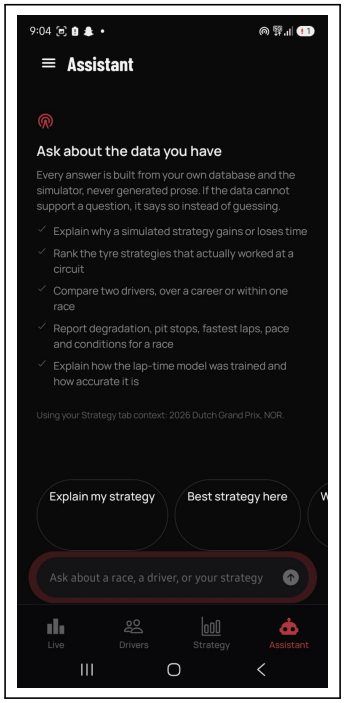
Screenshot 5.9: Strategy Tab - Module Picker



Screenshot 5.10: Stint Planner



Screenshot 5.11: Simulation Result



Screenshot 5.12: Grounded Assistant

CONCLUSIONS AND SCOPE FOR FUTURE WORK

OpenTyreF1 effectively uses open data and machine learning to turn Formula 1 telemetry into race-strategy decision support. It provides accurate, honestly evaluated predictions and grounded answers, and it makes them accessible through a fast mobile application. The system can be improved further by correcting the pit-lap rule, extending model coverage and adding live and multi-agent capabilities.

6.1 Conclusion

This project presented OpenTyreF1, an end-to-end system that integrates three heterogeneous public Formula 1 data sources into a single 237,308-record corpus and builds on it a lap-time model, a counterfactual strategy simulator, a Monte Carlo outcome estimator, seven pit-wall decision modules and a grounded assistant, delivered through a cross-platform mobile app.

The central finding is methodological. On data of this size and shape, the decisive choices are not about the learner but about which laps are admitted, how the target is parameterised and how evaluation is partitioned. Reparameterising the target improved MAE four-fold, while nearly doubling the feature count moved it by only 0.033 s. Under a strict whole-race holdout, the model reaches 0.893 s MAE and beats the strongest naive baseline by 0.270 s.

Beyond the numbers, OpenTyreF1 shows that a student-built system can be transparent about its own reliability: every simulation states its model source, every recommendation reports its sample size and every assistant answer cites its data. With further improvements, it has strong potential as a tool for fans, students and analysts, and as an open benchmark for research on motorsport analytics.

The project also gave the team practical experience across the full stack of a modern data product: integrating unreliable external APIs under rate limits, designing a relational schema, building a leakage-free machine-learning pipeline, writing a simulator, designing a REST API, building a cross-platform mobile app and deploying everything on free-tier infrastructure.

Equally valuable were the lessons in working with imperfect data. Several of the most important design decisions, such as logging and skipping unmatched sessions, reconstructing safety-car periods from free-text messages and measuring pit loss per circuit, came from problems discovered in the real data rather than from the original plan. Building the evaluation first, before tuning the model, meant that each of these decisions could

be judged by its measured effect.

6.2 Future Enhancements

- **Correct the Pit-Lap Rule:** Exclude in-laps per (driver, lap) pair and retrain, recovering about 38% more training data.
- **Better Attribution:** Use Shapley-value attribution [14] to separate tyre-age effects from the fuel-mass proxy.
- **Wider Coverage:** Complete the 2021 to 2022 backfill to evaluate the model across a regulation change.
- **Live Prediction:** Estimate the race median online so the accurate relative model can support live races.
- **Multi-Agent Simulation:** Add a field simulation in which rivals react to the user's strategy, together with an energy-deployment (ERS) estimate.
- **Richer Assistant:** Extend the grounded assistant with more intents and comparison queries while keeping every answer backed by data.
- **Notifications and Personalisation:** Add race-start reminders, favourite drivers and saved strategies.

Table 6.1 summarises these enhancements together with the benefit each is expected to bring.

Table 6.1: Future Enhancement Roadmap

Enhancement	Expected Benefit
Correct the pit-lap rule	About 38% more training data and a fairer clean-lap filter
Shapley-value attribution	Tyre-age effects separated from the fuel-mass proxy
2021-2022 backfill	Evaluation across a regulation change
Online race-median estimate	The accurate relative model available during live races
Multi-agent simulation and ERS	Rivals that react to the user's strategy
Richer assistant	More intents and comparisons, still backed by data
Notifications and personalisation	Race reminders, favourite drivers and saved strategies

Together, these enhancements would turn OpenTyreF1 from a retrospective analysis tool into a live strategy companion while keeping the honesty and grounding that define it.

6.3 Final Remarks

OpenTyreF1 began with a simple question: if Formula 1 data is now open, can a small team turn it into the kind of strategy insight that used to be available only on the pit wall? The answer from this project is yes, with one important condition. The value does not come from a more complex model; it comes from careful decisions about which data to trust, how to frame the prediction problem and how to measure success honestly.

These lessons extend beyond motorsport. Any domain with grouped, time-ordered data, such as sensor streams, patient visits or financial sessions, faces the same risks of leakage and the same temptation to report flattering scores. The approach followed here, of one shared data definition, a grouped holdout, a matched baseline and answers that cite their evidence, is a practical template for building trustworthy data products in those domains as well.

Finally, the project shows the value of building in the open. Because the data, the code, the model artifact and its measured metrics are kept together, every number in this report can be checked by running the system again. We hope that OpenTyreF1 will be useful both as a tool for people who follow Formula 1 and as a starting point for future students who want to work on motorsport analytics.

REFERENCES

[1] J. Bekker and W. Lotz, "Planning Formula One race strategies using discrete-event simulation," *Journal of the Operational Research Society*, vol. 60, no. 7, pp. 952–961, 2009.

[2] A. Heilmeier, M. Graf and M. Lienkamp, "A race simulation for strategy decisions in circuit motorsports," in *Proc. 21st IEEE Int. Conf. Intelligent Transportation Systems (ITSC)*, 2018, pp. 2986–2993.

[3] A. Heilmeier, A. Thomaser, M. Graf and J. Betz, "Virtual strategy engineer: Using artificial neural networks for making race strategy decisions in circuit motorsport," *Applied Sciences*, vol. 10, no. 21, art. 7805, 2020.

[4] X. Liu and A. Fotouhi, "Formula-E race strategy development using artificial neural networks and Monte Carlo tree search," *Neural Computing and Applications*, vol. 32, pp. 15191–15207, 2020.

[5] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[7] L. Grinsztajn, E. Oyallon and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[8] S. Kaufman, S. Rosset and C. Perlich, "Leakage in data mining: Formulation, detection, and avoidance," in *Proc. 17th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2011, pp. 556–563.

[9] D. R. Roberts et al., "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure," *Ecography*, vol. 40, no. 8, pp. 913–929, 2017.

[10] C. Newell, "Ergast Developer API: A web service for Formula One motor racing data." Available: http://ergast.com/mrd/

[11] Jolpica-F1, "An Ergast-compatible API for Formula 1 data." Available: https://github.com/jolpica/jolpica-f1

[12] OpenF1, "Open-source API for real-time and historical Formula 1 data." Available: https://openf1.org/

[13] P. Oehrly, "FastF1: A Python package for accessing and analysing Formula 1 results, schedules, timing data and telemetry." Available: https://github.com/theOehrly/Fast-F1

[14] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 4765–4774.

[15] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[16] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

International Journal for Research in Applied Science & Engineering Technology (IJRASET)
ISSN: 2321-9653; IC Value: 45.98; SJ Impact Factor: 7.538
Volume 14 Issue X Oct 2026- Available at www.ijraset.com

OpenTyreF1: A Data-Driven Mobile Platform for Formula 1 Telemetry Analysis, Tyre-Degradation Modelling and Counterfactual Race-Strategy Simulation

Atharva Joshi[1], Prachi Jain[2], Ankit Sawalakhe[3], Kripa Patil[4], Araju Yelekar[5], Devshree Khodke[6]
Department of Computer Science & Engineering, G H Raisoni University, Amravati, Maharashtra

Abstract: ***Formula 1 publishes one of the richest open telemetry streams in professional sport, yet most public tools built on it are descriptive: they show what happened and stop there. This paper presents OpenTyreF1, a full-stack system that turns that data into decision support on a mobile phone. The system integrates three public sources (Jolpica-F1/Ergast, OpenF1 and FastF1) into one normalised 15-table relational corpus of 237,308 records covering 492 races from 2002 to 2026. On top of this corpus it builds a shared clean-lap filter, a gradient-boosted (XGBoost) lap-time model, a counterfactual stint-strategy simulator, a 1,500-run Monte Carlo outcome estimator, seven pit-wall decision modules and a grounded question-answering assistant, all served by a FastAPI backend to an Expo/React Native app. Evaluated on 12 completely unseen races, the model achieves a mean absolute error of 0.893 s, RMSE of 1.205 s and R^2 of 0.978, beating the strongest naive baseline by 0.270 s. An ablation shows that predicting a lap's offset from its race median, rather than the absolute lap time, cuts error four-fold, a far larger effect than adding features. The results show that, for motorsport data of this size, careful lap admission, target design and leakage-free evaluation matter more than the choice of learner.***

Keywords: Formula 1, telemetry analytics, tyre degradation, XGBoost, race-strategy simulation, Monte Carlo methods, data leakage, mobile decision support, FastAPI, React Native.

I. INTRODUCTION

A modern Formula 1 car carries around three hundred sensors, and a meaningful part of that stream (lap times, tyre compounds and ages, pit-stop durations, race-control messages and trackside weather) is now published openly through community APIs. The raw material for quantitative race analysis is therefore no longer scarce. What is scarce is the analytical layer that turns it into decisions. Most fan and analyst tools plot what happened: a position chart, a stint diagram, a table of pit stops. The question a race strategist actually asks is counterfactual: "If this driver had pitted three laps later on the hard tyre, where would they have finished, and how sure can we be?" Answering it needs a calibrated lap-time model, an explicit rule for which laps represent true racing pace, a simulator that compares a hypothetical run against the field's real behaviour, and an honest statement of uncertainty.

OpenTyreF1 was built around exactly these requirements. Users can open any replayable race, build a tyre strategy, run it against what really happened, and receive an answer grounded in real lap data instead of generated text. The rest of this paper describes the architecture, the modules, the evaluation protocol and the measured results.

The key objectives of this research are:

To integrate heterogeneous public Formula 1 data sources into a single, reproducible relational corpus with idempotent, rate-limit-safe ingestion.

1) To define one shared clean-lap filter so that model training, published analytics and the mobile app always agree on what a representative lap is.
2) To train a lap-time prediction model and evaluate it with a leakage-free protocol that holds out whole races rather than random laps.
3) To build a counterfactual strategy simulator that converts predicted lap times into a finishing position against the real field.
4) To quantify uncertainty with Monte Carlo simulation and provide bounded pit-wall decision modules backed by historical base rates.
5) To deliver all of this through a fast, cross-platform mobile application with a grounded natural-language assistant.

©IJRASET: All Rights are Reserved

CERTIFICATES OF RESEARCH PUBLICATION

